BIBLIOTECA IA, SOFTWARE & COMPUTAÇÃO · LIVRO 6

Cultura, Comunidade e História

Comunidades e open-source, Linux, a preservação dos games, a história dos videogames e os pioneiros da computação.

Edição de outubro de 2026

Sobre esta coleção

> **Para começar**
> Esta coleção explica inteligência artificial, desenvolvimento de software e computação do zero. Cada capítulo começa em linguagem simples, com analogias do dia a dia, e aprofunda aos poucos. Os livros podem ser lidos separadamente.

O conteúdo reorganiza o que ensinam centenas de aulas e vídeos (do CS50 de Harvard a canais como IBM Technology, Anthropic e muitos outros), artigos técnicos de referência e o blog AkitaOnRails, complementados com pesquisa atualizada. Os vídeos aparecem pelo título, com link; cada capítulo termina com a tabela de fontes. Caixas *Você sabia?* trazem curiosidades e grandes nomes da tecnologia.

Os livros da coleção

1. Livro 1 — Fundamentos de IA e Modelos de Linguagem
2. Livro 2 — Construindo com LLMs
3. Livro 3 — Programando com IA
4. Livro 4 — Engenharia de Software e Ciência da Computação
5. Livro 5 — IA, Negócios e Sociedade
6. Livro 6 — Cultura, Comunidade e História

> **Datas**
> IA muda depressa. Preços, planos, benchmarks e nomes de modelos refletem outubro de 2026. Figuras marcadas como "ilustrativas" mostram a ideia, não medições.

Sumário

1. Comunidades, Linux e o ecossistema aberto que alimenta a IA..... 4
2. IA e a ressignificação dos games..... 16
3. Uma volta no tempo: dos primeiros jogos à IA nos games..... 27
4. E se Ada Lovelace e Alan Turing vissem a computação hoje? 43

CAPÍTULO 1 ·

Comunidades, Linux e o ecossistema aberto que alimenta a IA

> *Toda IA que escreve código ou responde a uma dúvida aprendeu com gente: com perguntas e respostas no Stack Overflow, discussões no Reddit, blogs, repositórios no GitHub, aulas no YouTube e fóruns em dezenas de línguas. E quase tudo isso roda sobre o Linux, um sistema criado por um estudante finlandês como "hobby". Este capítulo mostra como esse ecossistema aberto se formou, por que ele é a matéria-prima da IA, como a própria IA ameaça esvaziá-lo e o que cada leitor pode fazer para mantê-lo vivo. Dados verificados em 31/10/2026.*

Para começar

Imagine um mural de bairro onde os moradores escrevem perguntas e respostas; com os anos, ele vira uma enciclopédia. Uma empresa fotografa o mural, treina um robô com as fotos e o coloca na porta, respondendo tudo na hora. As pessoas param de escrever no mural. De onde o robô vai aprender as respostas novas?

Por que comunidades importam na era da IA

Para começar

Uma **comunidade técnica** é qualquer lugar onde pessoas trocam conhecimento de forma pública e pesquisável: fórum, subreddit, repositório aberto, blog. Por vinte anos, foi assim que programadores aprenderam o que não estava no livro, e foi com esse acervo que os LLMs (Livro 1, Capítulo 2 (LLMs e Transformers)) foram treinados.

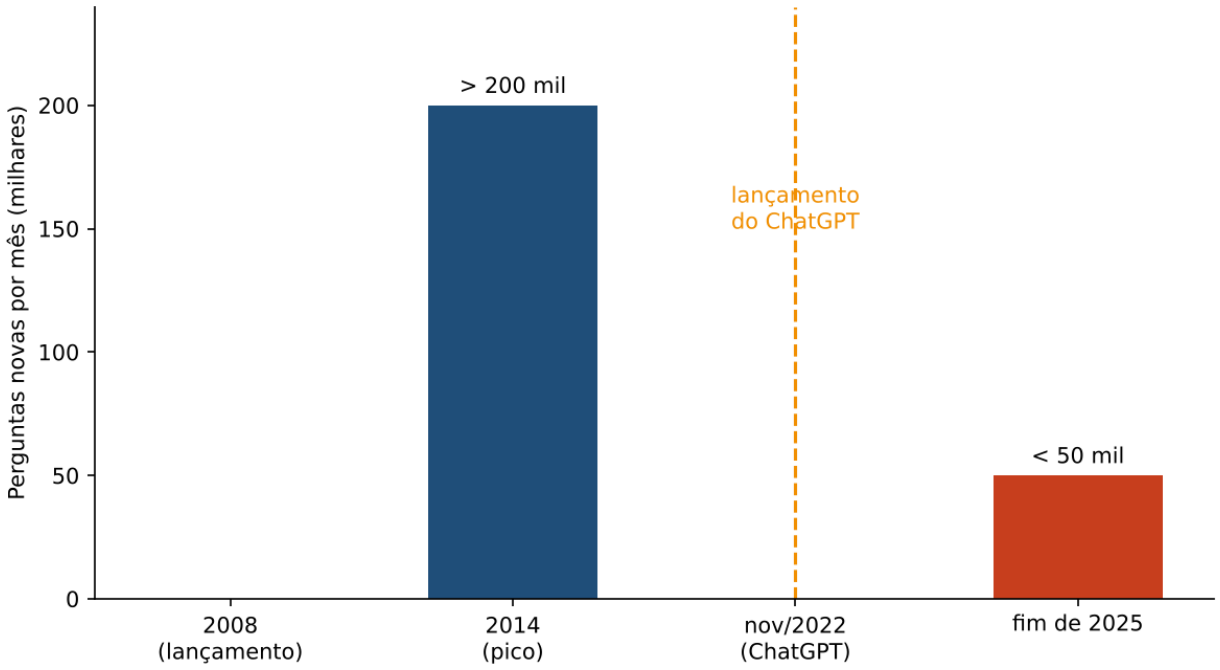
Stack Overflow: a enciclopédia de respostas

Lançado em 2008 por Jeff Atwood e Joel Spolsky, o Stack Overflow trocou os fóruns caóticos por um formato simples: uma pergunta, várias respostas, votos que colocam a melhor no topo e um sistema de **reputação** que recompensa quem ajuda. Perguntas duplicadas são fechadas e apontadas para a original, e respostas podem ser editadas por usuários com reputação suficiente, como num wiki. Por mais de uma década, quase toda busca por uma mensagem de erro terminava ali. O acervo, porém, era sustentado por uma minoria: já em 2013, 75% dos usuários tinham feito uma única pergunta na vida e só 8% tinham respondido mais de cinco.

Um detalhe jurídico fez diferença para a IA: as contribuições são publicadas sob licença Creative Commons (CC BY-SA) e a empresa publicava *data dumps* completos. Eram milhões de pares "pergunta real → resposta revisada e votada", o conjunto ideal para treinar modelos de código. Um efeito colateral, discutido no Livro 3, Capítulo 1 (Programando com IA), é que os modelos tendem a sugerir as bibliotecas e padrões populares em respostas públicas.

> **Você sabia?**
>
> O nome "Stack Overflow" foi escolhido por votação entre leitores dos blogs dos fundadores. Ele vem do *estouro de pilha*, erro que ocorre quando a pilha de chamadas cresce além da memória reservada, como numa recursão sem condição de parada; a aula CS50x em Português - Aula 4 - Memória mostra o desenho da memória que explica o nome.

O site passou de **200 mil perguntas novas por mês em 2014**, seu pico. A queda começou antes da IA (as dúvidas fáceis já estavam respondidas e a moderação rígida afastava iniciantes), mas o ChatGPT, em novembro de 2022, a acelerou: no fim de 2025 eram **menos de 50 mil** perguntas por mês, segundo o PPC Land. Pela primeira vez havia uma alternativa que respondia na hora, sem julgamento e adaptada ao código exato do usuário. Em 2023, uma mudança na política sobre conteúdo gerado por IA levou a uma greve de moderadores voluntários: mais de 70% dos moderadores do site chegaram a parar, até um acordo em agosto.



Perguntas novas por mês no Stack Overflow: lançamento (2008), pico (mais de 200 mil em 2014) e fim de 2025 (menos de 50 mil), com o lançamento do ChatGPT marcado. Mostramos só os pontos com valores publicados. Fonte: elaboração própria com dados de PPC Land (jan/2026).

A empresa transformou o acervo em produto: em 2024 fechou acordos de dados com o Google Cloud e com a OpenAI. Usuários que apagaram respostas em protesto tiveram as edições revertidas, já que a licença não permite "retirar" o que foi doado. O episódio expôs a tensão central do capítulo: o conhecimento foi doado à comunidade, mas o valor

econômico foi capturado por empresas. Ainda assim, a pesquisa de desenvolvedores de 2025 mostrou que 84% usam ou planejam usar IA, mas só 3,1% confiam "muito" nela: o site virou menos um lugar para perguntar e mais um lugar para verificar. A mesma pesquisa registrou que 66% dos desenvolvedores se frustram com respostas de IA "quase certas, mas não totalmente", justamente o tipo de erro que a revisão por pares da comunidade costuma pegar.

Reddit e o laboratório do r/LocalLLaMA

Se o Stack Overflow é uma enciclopédia, o Reddit é uma praça: milhares de *subreddits*, cada um com moderadores voluntários e regras próprias. O conhecimento é menos "a resposta certa" e mais experiência vivida ("testei as três placas e a do meio esquentou demais"), opinião e comparação, o que ensina aos modelos a linguagem natural real, com gírias e discordâncias. Em 2023 a empresa passou a cobrar caro pela API, em parte para não entregar o acervo de graça às empresas de IA. O protesto que se seguiu fechou milhares de subreddits por dias. Em 2024 vieram os acordos de licenciamento com o Google (cerca de US$ 60 milhões por ano) e com a OpenAI.

> **Atenção: lixo entra, lixo sai**
> Em 2024, os resumos por IA da busca do Google sugeriram usar cola para o queijo grudar na pizza, eco de uma piada antiga do Reddit. O vídeo How to Make AI More Accurate: Top Techniques for Reliable Results, da IBM Technology, usa o caso para introduzir técnicas como RAG e ajuste do *system prompt*. Dados de comunidade precisam de curadoria.

Para quem trabalha com IA, o subreddit mais importante é o **r/LocalLLaMA**, criado em 2023 após o vazamento dos pesos do primeiro LLaMA. Ele virou o ponto de encontro de quem roda modelos localmente: primeiras medições de modelos abertos, receitas de quantização (Livro 3, Capítulo 2 (Hardware e IA local)) e comparações de placas. Funciona como um laboratório distribuído, com milhares de pessoas testando a mesma coisa em hardwares diferentes e publicando o que deu certo e o que deu errado. É comum que um modelo chinês ou europeu seja "descoberto" pelo público ocidental numa discussão ali antes de sair em qualquer jornal.

Blogs pessoais: conhecimento com nome e sobrenome

Um texto longo e assinado tem algo que uma resposta de IA não tem: **responsabilidade**, porque o autor põe a reputação em jogo. Três exemplos:

- **Fabio Akita (AkitaOnRails).** Um dos blogs técnicos mais antigos do Brasil, hoje conhecido por testes reproduzíveis de modelos abertos, como LLM Benchmark v4: o veredito de open source (até out/2026): comparações independentes, em português, com metodologia exposta.
- **Simon Willison.** Cocriador do Django, mantém em simonwillison.net um dos diários mais lidos sobre LLMs e popularizou o termo *prompt injection*. É o modelo de quem "aprende em público": documenta quase diariamente o que testa, com código e exemplos, até em notas curtas.

- **Andrej Karpathy.** Cofundador da OpenAI, mantém o blog karpathy.github.io e aulas no YouTube em que implementa redes neurais do zero; cunhou o termo *vibe coding* em 2025. Projetos minimalistas como o nanoGPT ensinaram uma geração a entender por dentro o que os grandes laboratórios faziam.

O ponto em comum: os três publicam abertamente, com código, e por isso viram referência. Todo esse material também está nos dados de treinamento dos modelos.

GitHub e o código como bem comum

O GitHub, de 2008 e comprado pela Microsoft em 2018, é construído sobre o **Git**, criado por Linus Torvalds em 2005 para o núcleo do Linux. Seu conceito central é o **pull request**: você copia o projeto, faz a mudança e pede ao mantenedor que a incorpore, com revisão pública (Livro 3, Capítulo 4 (Desenvolvimento seguro e Pull Request)). Segundo o Octoverse 2025, a plataforma passou de 180 milhões de desenvolvedores, o TypeScript virou a linguagem número 1 (tipos ajudam agentes a errar menos), mais de 1,1 milhão de repositórios usam SDKs de LLM e o Brasil é o 4º país em número de desenvolvedores.

O código aberto é um bem comum no sentido literal: qualquer um pode usar, estudar, modificar e redistribuir, conforme a licença. As licenças se dividem em duas famílias. As **permissivas** (MIT, Apache 2.0, BSD) permitem até incorporar o código em produtos fechados, mantendo o aviso de autoria. As **copyleft** (GPL, AGPL) exigem que obras derivadas distribuídas mantenham a mesma licença. O Linux usa a GPLv2, e isso ajuda a explicar por que concorrentes como Intel, AMD e Google contribuem para o mesmo núcleo: nenhum pode fechá-lo para si.

A lógica vale para artefatos novos: no vídeo Compartilhando minhas Skills | Projeto Open Source, Fernanda Kipper cria um repositório aberto de *skills* para agentes (Livro 2, Capítulo 4 (Agentes e MCP)) e defende que o valor está em deixar a comunidade contribuir e melhorar.

Hugging Face e o ecossistema chinês

A **Hugging Face**, fundada em 2016 como aplicativo de chatbot, mudou de rumo ao publicar em código aberto uma implementação do BERT, o modelo de linguagem do Google. Daí nasceu a biblioteca **Transformers**, padrão para rodar modelos em Python, e o **Hub**, repositório público de modelos e conjuntos de dados com páginas descritivas (*model cards*). É um terreno neutro: Meta, Google, Mistral, Alibaba, DeepSeek e pessoas físicas publicam pesos ali, e em dias a comunidade gera versões quantizadas (GGUF) e ajustadas. Esse ciclo, que leva dias, seria impossível com modelos fechados. Regra prática: leia o *model card* (licença, dados, limitações) antes de baixar e prefira organizações conhecidas.

A China tem um ecossistema técnico paralelo. A **CSDN**, de 1999, se descreve como "Stack Overflow + GitHub para desenvolvedores chineses" e afirma reunir mais de 47 milhões de desenvolvedores. A Alibaba publica a família **Qwen** com pesos abertos e mantém o **ModelScope**, equivalente ao Hub; a **DeepSeek** publicou em 2025 o DeepSeek-R1 sob licença MIT, mostrando que um modelo de raciocínio de ponta podia custar bem me-

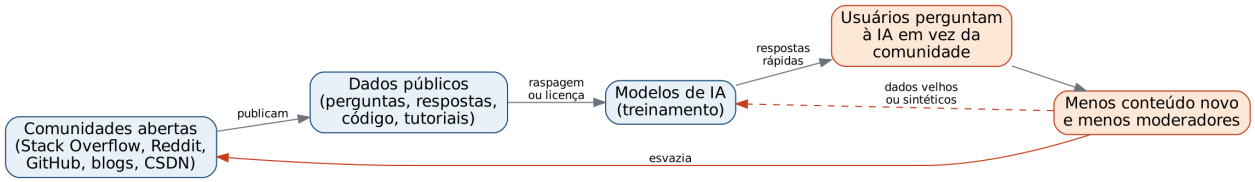
nos do que se supunha. Comunidades como a CSDN, o Zhihu e grupos de WeChat são onde esses modelos são discutidos, testados e adaptados, e muitos dados de treino em chinês vêm desses sites. Nos testes independentes de 2026, modelos chineses abertos aparecem entre os melhores para uso local.

> **Você sabia?**
> Em 2011, dados de cerca de 6 milhões de usuários da CSDN vazaram, e as senhas estavam guardadas em texto puro, sem *hash*. O caso ajudou a mudar as práticas de segurança na internet chinesa (Livro 4, Capítulo 4 (Eng. Reversa e Segurança) explica por que isso é inaceitável).

Comunidade	Tipo de conhecimento	Relação com a IA
Stack Overflow	Perguntas e respostas votadas	Vendeu o acervo a Google e OpenAI; recebe cada vez menos perguntas
Reddit	Conversa, experiência, opinião	Vendeu o acervo a Google e OpenAI; no r/LocalLLaMA, gente comum testa modelos em casa
Blogs pessoais	Análises longas e assinadas	Autores testam modelos por conta própria e explicam o resultado
GitHub	Código, issues, revisões	Os assistentes de código aprenderam lendo esse código
Hugging Face	Pesos e datasets	Lugar onde todos publicam modelos abertos para download
CSDN e ModelScope	Blogs, fórum e modelos em chinês	Onde se discutem modelos chineses como Qwen e DeepSeek

O paradoxo: a IA comeu o mural

As comunidades produziram de graça o conhecimento que tornou os modelos úteis; os modelos reduzem o motivo para alguém perguntar ou responder nelas. Sem perguntas, não surge conhecimento público sobre as bibliotecas e erros de amanhã, e os próximos modelos terão de aprender com documentação, código e dados sintéticos. Treinar repetidamente com dados gerados por modelos tende a degradar a diversidade e a qualidade, fenômeno chamado de **colapso de modelo** (*model collapse*). O conhecimento novo continua surgindo de gente lidando com sistemas novos; a questão é se ele fica guardado em conversas privadas com assistentes ou volta para o mural público, onde pessoas e máquinas podem aprender com ele.



O ciclo do paradoxo: comunidades geram os dados que treinam os modelos; os modelos afastam os usuários das comunidades, que produzem menos conteúdo, e o próximo treino fica com dados mais velhos ou sintéticos. Fonte: elaboração própria.

É uma **tragédia dos comuns**: um recurso compartilhado é consumido por todos, mas ninguém tem incentivo individual para repô-lo. As saídas conhecidas:

1. **Pagar a quem produz**, via acordos de licenciamento ou patrocínio direto (GitHub Sponsors, Open Collective).

2. **Atribuição**: assistentes que citam a fonte com link devolvem tráfego e reduzem alucinações (Livro 2, Capítulo 3 (RAG)).
3. **Devolver o que se aprende**: publicar a solução de um problema não documentado que você resolveu com IA.
4. **Comunidades com outro papel**: oferecer o que a IA não oferece, como experiência vivida, revisão humana, mentoria e pertencimento. O Stack Overflow está virando lugar de verificação; o Discord e os encontros presenciais, de convivência.

A importância do Linux

Para começar

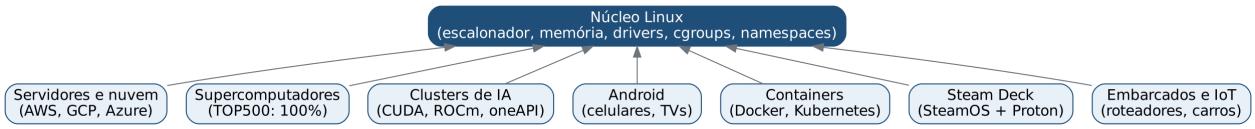
Um **sistema operacional** fica entre o hardware e os aplicativos: reparte o processador, guarda arquivos, conversa com a placa de vídeo e a rede. Em casa você talvez use Windows ou macOS, mas quase todo o resto do mundo digital, do servidor ao supercomputador que treina IA, roda **Linux**. Ele é o encanamento da cidade: ninguém vê, tudo depende dele.

Você sabia?

Em 25 de agosto de 1991, Linus Torvalds, então com 21 anos, anunciou na Usenet que fazia "um sistema operacional (gratuito), só um hobby, não vai ser grande e profissional como o GNU". Em 1992 adotou a licença GPL, e o núcleo passou a ser combinado com as ferramentas GNU.

Estritamente, "Linux" é o **núcleo** (*kernel*). O que se instala é uma **distribuição** (Ubuntu, Debian, Fedora, Arch etc.), que junta o núcleo a ferramentas, gerenciador de pacotes e interface. Todas compartilham o mesmo modelo: arquivos, permissões, processos e um terminal. Em 2005, para gerenciar as contribuições de milhares de desenvolvedores, Torvalds criou o Git em poucas semanas. O desenvolvimento do núcleo, com uma versão nova a cada nove ou dez semanas e milhares de pessoas e centenas de empresas a cada versão, é o maior exemplo de bem comum de software mantido por concorrentes.

Onde o Linux está



O mesmo núcleo Linux na base de servidores e nuvem, supercomputadores, clusters de IA, Android, containers, Steam Deck e dispositivos embarcados. Fonte: elaboração própria.

- **Servidores e nuvem.** A maioria dos servidores web e das máquinas virtuais em nuvem roda Linux; a Microsoft mantém o WSL, que coloca um Linux real dentro do Windows.
- **Supercomputadores.** Desde novembro de 2017, **100% da lista TOP500** usa Linux.
- **IA.** Os clusters que treinam modelos de fronteira rodam Linux, e as pilhas de IA (CUDA, ROCm, oneAPI, PyTorch, vLLM) são desenvolvidas e testadas primeiro nele.

- **Android**, o sistema de celular mais usado do mundo, roda sobre um núcleo Linux modificado.
- **Containers.** Docker e Kubernetes (Livro 4, Capítulo 1 (Eng. Software)) se apoiam em dois recursos do núcleo: **namespaces**, que isolam o que um processo enxerga, e **cgroups**, que limitam o que ele consome.
- **Games.** O Steam Deck, lançado pela Valve em 2022, roda o SteamOS, baseado em Arch Linux, e executa jogos de Windows pela camada de compatibilidade **Proton**, baseada no Wine, que traduz chamadas do Windows (inclusive DirectX para Vulkan) em tempo real. O aparelho provou que "Linux não roda jogos" deixou de ser verdade para a maior parte do catálogo.
- **Embarcados.** Roteadores, televisores, carros e equipamentos industriais usam versões enxutas de Linux.

Drivers de GPU e por que todo dev de IA precisa saber Linux

O **driver** da placa de vídeo é por onde bibliotecas como PyTorch enviam trabalho à GPU. A NVIDIA publicou em 2022 módulos de núcleo abertos, embora o CUDA continue proprietário; AMD (`amdgpu`, ROCm) e Intel (`i915`, `xe`, oneAPI) mantêm drivers abertos. Na prática, versão do núcleo, do driver e da biblioteca de computação precisam ser compatíveis; um erro comum é instalar o PyTorch sem verificar qual versão de CUDA ou ROCm ele espera. Alguns comandos de diagnóstico resolvem boa parte das dúvidas:

```
uname -r                  # versão do núcleo
lspci | grep -iE "vga|3d"  # GPUs que o sistema enxerga
nvidia-smi                # GPUs NVIDIA: driver, CUDA, memória
python -c "import torch; print(torch.__version__, torch.cuda.is_available())"
```

Há três motivos para saber Linux: o código quase certamente vai rodar em Linux (servidor, container, CI); os agentes de código trabalham pelo terminal, e você precisa ler o que eles executam; e a documentação de IA assume Linux como padrão. Não é preciso virar administrador de sistemas. A aula CS50x em Português - Aula 1 - C já apresenta os comandos básicos, e o roteiro abaixo cobre o dia a dia:

Tema	O que aprender	Por que importa em IA
Navegação e arquivos	`ls`, `cd`, `find`, `grep`, `less`	Achar um modelo baixado ou o registro de um erro
Permissões	`chmod`, `chown`, `sudo`, grupos	Liberar a placa de vídeo sem dar acesso a tudo
Processos e recursos	`ps`, `htop`, `kill`, `free`, `df`	Descobrir por que o computador travou
Pacotes e ambientes	`apt`/`dnf`, `venv`, `uv`	Instalar a versão certa de cada biblioteca
Rede e acesso remoto	`ssh`, `scp`, `curl`	Usar de casa um computador com GPU alugado
Serviços e containers	`systemctl`, `journalctl`, Docker/Podman	Deixar um modelo respondendo dia e noite
Shell script	Variáveis, laços, pipes (`|`), redirecionamento	Repetir tarefas sem digitar tudo e ler o que o agente faz

> **Dica**
> Não é preciso formatar o computador: no Windows, o WSL2 instala um Ubuntu completo (`wsl --install`) com acesso à GPU; no macOS, o terminal já é Unix.

Os serviços do Google nesse cenário

> **Para começar**
> O Google é ao mesmo tempo um dos maiores consumidores de conhecimento público (a busca indexa a web) e um dos maiores produtores de conhecimento aberto. Muitos tijolos da IA atual, inclusive dos concorrentes, saíram de lá.

YouTube. Comprado em 2006, virou a maior escola do mundo, com cursos completos gratuitos, como o CS50 de Harvard em português, e aulas de pesquisadores como Karpathy. Canais brasileiros ensinam programação, infraestrutura e IA a milhões de pessoas que não teriam acesso a um curso pago, e este livro foi construído em boa parte a partir de aulas publicadas ali, citadas pelo título ao longo dos capítulos. A coleta de legendas e transcrições por empresas de IA sem autorização gerou disputas públicas em 2024. O YouTube também ensina máquinas: o Gemini consegue "assistir" a um vídeo e responder perguntas sobre ele. A qualidade varia muito: prefira canais com código e resultados reproduzíveis e confira a data dos tutoriais.

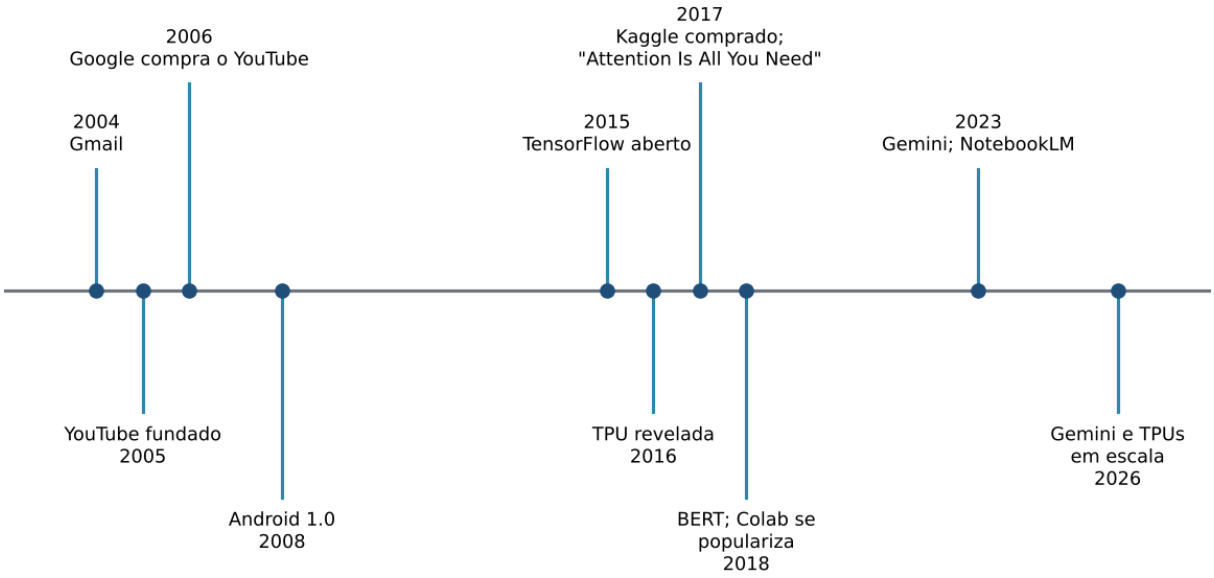
"Attention Is All You Need". Em 2017, oito pesquisadores do Google publicaram o artigo que propôs o **Transformer**, arquitetura baseada inteiramente em **atenção**, em que cada palavra "olha" para as outras para decidir o que é relevante (Livro 1, Capítulo 2 (LLMs e Transformers)). Como foi publicado abertamente, permitiu o GPT (o "T" é de Transformer) e os modelos de todos os concorrentes. É um caso raro: a invenção central de uma revolução foi dada ao mundo por quem a criou.

> **Você sabia?**
> O título "Attention Is All You Need" brinca com a música "All You Need Is Love", dos Beatles. Todos os oito autores acabaram deixando o Google, vários para fundar empresas de IA.

Ferramentas e hardware. O Google mantém o TensorFlow, aberto desde 2015 e primeira biblioteca de aprendizado profundo de uso industrial amplo, além de JAX e Keras; projeta as **TPUs**, chips para as multiplicações de matrizes das redes neurais e principal alternativa às GPUs da NVIDIA; oferece o **Colab**, cadernos no navegador com GPU gratuita (com limites); e mantém o **Kaggle**, comunidade de competições de ciência de dados com conjuntos públicos, cadernos compartilhados e um placar que funciona como currículo. Para milhões de estudantes sem placa de vídeo, o Colab foi a primeira chance de treinar uma rede neural.

Android, Gemini e NotebookLM. O Android é código aberto (AOSP) sobre o núcleo Linux; graças a ele, bilhões de pessoas tiveram seu primeiro computador num celular barato. É cada vez mais plataforma de inferência local (Gemini Nano, Gemma). O Gemini é a família de modelos de fronteira, multimodal; o Gemma, a versão com pesos abertos. O

NotebookLM trabalha só sobre as fontes que o usuário carrega (PDFs, vídeos, páginas) e responde com citações para o trecho exato: na prática, um RAG pronto para leigos, útil para transformar aulas em material de revisão, desde que se confiram as citações.



Linha do tempo de produtos e contribuições do Google relevantes para o ecossistema de IA e conhecimento aberto. Fonte: elaboração própria com base em comunicados públicos do Google e no artigo de Vaswani et al. (2017).

Por que mais gente vai criar o próprio software

> **Para começar**
> Fazer um aplicativo exigia uma equipe, como construir uma casa exigia uma construtora. Com IA, plataformas prontas e distribuição gratuita, muita gente consegue construir uma casa pequena sozinha. As construtoras continuam existindo, mas o número de casas se multiplica.

Quatro forças explicam a mudança: o **custo de execução caiu** (modelos abertos de bom nível já rodam em hardware doméstico); **agentes** como Claude Code, Codex e Cursor fazem o trabalho repetitivo (Livro 3, Capítulo 3 (AI-Driven Engineer)), sendo, como argumenta Fabio Akita em LLMs + Agentes = competente o suficiente, suficientes para muitas tarefas quando alguém sabe avaliar o resultado; ferramentas **low-code** geram o que falta; e a **distribuição** por GitHub, lojas e itch.io é quase gratuita. O resultado é o **software pessoal**: o controle financeiro da família ou o bot que avisa quando um preço cai, que antes não compensava e agora custa uma tarde.

> **Atenção**
> Um agente ter escrito o código não transfere a responsabilidade: senhas expostas e dependências desatualizadas continuam sendo problema de quem publicou (Livro 3, Capítulo 4 (Desenvolvimento seguro e Pull Request)).

O open-source tende a crescer junto: os modelos abertos (Qwen, DeepSeek, Llama, Mistral, Gemma) ficaram competitivos; governos e empresas buscam **soberania** sobre dados e serviços críticos; o custo marginal de rodar um modelo próprio é energia e máquina; e código aberto pode ser auditado por todos. Essa auditoria não é garantia, como mostrou a porta dos fundos inserida no pacote xz em 2024, mas foi justamente um olhar externo que a descobriu. Há ainda a **regulação**: o AI Act europeu dá tratamento diferenciado a modelos abertos em algumas obrigações e exige transparência, mais fácil de cumprir com sistemas abertos. O contraponto: "aberto" nem sempre é o mesmo. A *Open Source Initiative* publicou em 2024 uma definição de "IA de código aberto" que exige informação suficiente sobre os dados, e poucos modelos a cumprem. Muitos modelos publicam só os pesos, sem dados nem código de treino, e por isso o termo preciso é **pesos abertos** (*open weights*). Além disso, mantenedores sofrem com sobrecarga, agravada por contribuições de baixa qualidade geradas por IA.

Como participar: guia prático

> **Para começar**
> Participar não exige ser especialista. A maioria das contribuições úteis é pequena: corrigir a documentação, reportar um bug com detalhes, responder a uma dúvida que você teve semana passada. É um mutirão: basta carregar alguns tijolos.

Consumir e devolver o mínimo. Vote na resposta que resolveu seu problema, deixe uma estrela no repositório ou diga em que versão funcionou. Quando usar IA para resolver algo não documentado, publique a solução depois.

Perguntar e reportar bem. Saber perguntar também melhora seus prompts (Livro 2, Capítulo 1 (Prompts na prática)). Antes de abrir uma pergunta ou issue:

1. Pesquise se já existe, incluindo issues fechadas.
2. Descreva o objetivo, não só o erro.
3. Inclua versões, o comando exato e a mensagem de erro completa como texto.
4. Monte um **exemplo mínimo reproduzível** e diga o que já tentou.

Primeira contribuição de código. Procure etiquetas como `good first issue`, leia o `README`, o `CONTRIBUTING.md` e o código de conduta. O fluxo típico no GitHub:

```
gh repo fork dono/projeto --clone
cd projeto
git switch -c corrige-doc-instalacao
# ... edita e roda os testes do projeto ...
git commit -am "docs: corrige comando de instalação"
git push -u origin corrige-doc-instalacao
gh pr create --fill
```

> **Dica: IA sim, lixo não**
>
> Usar um agente para entender um projeto é ótimo; abrir dezenas de pull requests gerados sem testar é uma das maiores queixas dos mantenedores. Vários projetos passaram a exigir que o autor declare o uso de IA. Só envie o que você consegue explicar linha por linha, mantenha a mudança pequena e responda às revisões com educação.

Ler as discussões. Siga dois ou três blogs e canais de referência e leia as discussões, não só os títulos; comunidades como o r/LocalLLaMA ensinam muito só pela leitura.

Publicar e ensinar. Publique projetos com licença clara (MIT ou Apache 2.0 para máxima adoção; GPL ou AGPL para que melhorias voltem) e um README com instalação simples. Escreva sobre o que aprendeu, publique modelos com *model card* honesto, participe de meetups e apoie financeiramente os projetos dos quais depende.

Boas maneiras e segurança. Seja gentil com iniciantes: a fama de hostilidade afastou gente do Stack Overflow antes mesmo da IA. Nunca publique chaves de API, senhas ou arquivos `.env`. Confira autor e atualização de pacotes e modelos antes de instalar, lembre que ataques à cadeia de suprimentos (pacotes falsos com nomes parecidos, modelos com código embutido) são reais, prefira formatos seguros como `safetensors` e respeite as licenças (Livro 5, Capítulo 3 (Propriedade intelectual)).

Fontes deste capítulo

Dados verificados em 31/10/2026.

Título	URL	Canal / autor	Ponto principal
CS50x em Português - Aula 4 - Memória	https://www.youtube.com/watch?v=DweS4KseeDE	CS50	Heap, stack e origem do "stack overflow"
CS50x em Português - Aula 1 - C	https://www.youtube.com/watch?v=eepzDoovmAw	CS50	Comandos básicos de terminal Linux
Compartilhando minhas Skills | Projeto Open Source	https://www.youtube.com/watch?v=2y2e3Y4oMFo	Fernanda Kipper	Repositório aberto de skills para agentes
How to Make AI More Accurate: Top Techniques for Reliable Results	https://www.youtube.com/watch?v=pNbU1vGkIK4	IBM Technology	"Cola na pizza" e técnicas de precisão
LLM Benchmark v4: o veredito de open source (até out/2026)	https://akitaonrails.com/2026/10/22/llm-benchmark-v4-o-veredito-de-open-source-ate-out-2026/	AkitaOnRails	Panorama dos modelos abertos em 2026
LLMs + Agentes = competente o suficiente	https://akitaonrails.com/2026/10/12/llms-agentes-competente-o-suficiente/	AkitaOnRails	Agentes de código bastam para muitas tarefas
Stack Overflow traffic collapses as AI tools reshape how developers code	https://ppc.land/stack-overflow-traffic-collapses-as-ai-tools-reshape-how-developers-code/	PPC Land	Queda de perguntas no Stack Overflow
Stack Overflow and Google Cloud Announce Strategic Partnership	https://stackoverflow.co/company/press/archive/google-cloud-strategic-gen-ai-partnership/	Stack Overflow	Acordo de dados com Google Cloud
Stack Overflow to provide coding knowledge to OpenAI models	https://siliconangle.com/2024/05/06/stack-overflow-provide-coding-knowledge-openai-models/	SiliconANGLE	Acordo de dados com a OpenAI
Reddit confirms Google licensing deal	https://www.theregister.com/2024/02/22/reddit_google_license_ipo_altman/	The Register	Acordo Reddit–Google, ~US$ 60 mi/ano
Reddit hawks your posts to our AI overlords again, this time to OpenAI	https://www.pcgamer.com/software/ai/reddit-hawks-your-posts-to-our-ai-overlords-again-this-time-to-openai	PC Gamer	Acordo Reddit–OpenAI (mai/2024)

Título	URL	Canal / autor	Ponto principal
Octoverse 2025	https://github.blog/news-insights/octoverse/octoverse-a-new-developer-joins-github-every-second-as-ai-leads-typescript-to-1/	GitHub	180 mi de devs; TypeScript nº 1
CSDN: Who we are	https://devpress.csdn.net/csdn_en	CSDN	47 mi+ de desenvolvedores chineses
Chinese Software Developer Network	https://en.wikipedia.org/wiki/Chinese_Software_Developer_Network	Wikipedia	Fundação em 1999; vazamento em 2011
Attention Is All You Need	https://arxiv.org/abs/1706.03762	Vaswani et al. (Google), 2017	Artigo que propôs o Transformer
TOP500: estatísticas por sistema operacional	https://www.top500.org/statistics/list/	TOP500	Linux em 100% do TOP500
Simon Willison's Weblog	https://simonwillison.net/	Simon Willison	Diário técnico sobre LLMs
Andrej Karpathy blog	https://karpathy.github.io/	Andrej Karpathy	Textos didáticos sobre redes neurais

CAPÍTULO 2 ·

IA e a ressignificação dos games

> *Nos últimos anos, comunidades de fãs reconstruíram o código-fonte de clássicos do Nintendo 64, criaram versões nativas para PC, traduziram jogos para o português e passaram a usar modelos de linguagem para acelerar esse trabalho. Ao mesmo tempo, empresas desligaram servidores, retiraram jogos das lojas e deixaram claro que o jogador "comprou" uma licença, não uma cópia. Este capítulo explica as técnicas em nível conceitual, apresenta casos públicos, discute o que as empresas podem aprender e fecha com um panorama ético e legal. Fatos, leis e números foram verificados em 30/10/2026.*

Para começar

Um jogo não é só uma gravação: é um **programa** escrito para uma máquina que já não é fabricada. Se a máquina some e ninguém relança o jogo, ele deixa de existir na prática. "Ressignificar" é dar nova vida a esses jogos, em outro aparelho, em outro idioma ou com gráficos melhores, e a IA entrou nesse movimento como acelerador.

O vocabulário básico: emular, decompilar, recompilar, portar

Para começar

Imagine um bolo pronto cuja receita (o código-fonte) ficou com a confeitaria. Você pode construir um forno idêntico ao original (emulação), reescrever uma receita que dê exatamente o mesmo bolo (decompilação) ou traduzir as instruções do forno antigo para um moderno (recompilação). Com a receita em mãos, dá para adaptá-la a outra cozinha (port).

Um jogo é escrito em uma linguagem legível, como C, e um **compilador** o transforma em **código de máquina**, as instruções numéricas que o processador executa. No cartucho ou disco vão só esse código e os **assets** (gráficos, sons, textos). Compilar é quase uma estrada de mão única: nomes de variáveis, comentários e organização desaparecem, e voltar a algo legível é trabalho de reconstrução, não de "desfazer".

Técnica	O que faz	Precisa do jogo original?
Emulação	Um programa imita o hardware do console; o jogo roda sem modificação.	Sim, a ROM ou o disco.
Engenharia reversa	Termo guarda-chuva: estudar um programa pronto para entender como funciona.	Sim, para estudar.
Decompilação	Reconstruir, a partir do binário, um código-fonte em linguagem de alto nível.	Sim, para comparar.
Recompilação estática	Traduzir de antemão o código de máquina para outra linguagem e gerar um executável nativo.	Sim; os assets vêm da cópia do jogador.
Port	Fazer o jogo rodar nativamente em outra plataforma.	Nos ports de fãs, quase sempre sim.

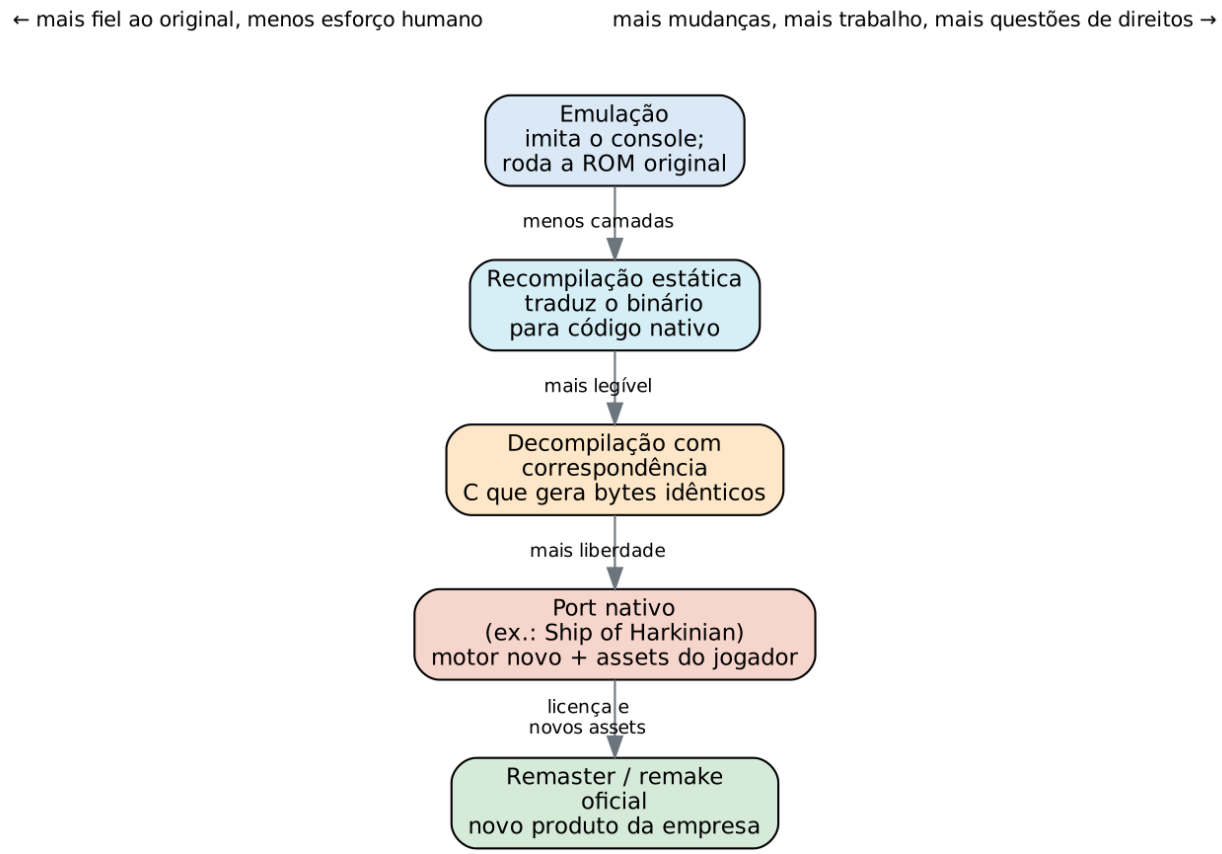
No mercado, **remaster** é o relançamento com melhorias técnicas e **remake** é o jogo refeito, muitas vezes do zero.

Um emulador é um programa que finge ser o console. Ele lê cada instrução do jogo e reproduz o efeito que ela teria no processador, na placa de vídeo e no chip de som originais. A grande vantagem é a fidelidade: o jogo roda como foi lançado, sem ninguém ter mexido nele. A desvantagem é o custo: imitar outro hardware gasta muito mais processamento do que rodar um programa nativo, e certos efeitos (como os truques gráficos do Nintendo 64) são difíceis de reproduzir com perfeição. Melhorias como resolução maior ou 60 quadros por segundo dependem de "remendos" externos, porque o emulador não sabe o que o jogo "quer" fazer, só executa as instruções.

Em vez de interpretar instrução por instrução durante o jogo, a **recompilação estática** lê o código de máquina inteiro de antemão e o converte para outra linguagem (frequentemente C), que depois é compilada para o PC. O resultado é um executável nativo, muito mais rápido que um emulador, que ainda carrega a lógica original do jogo. O código gerado não é bonito nem legível como o de um programador, mas funciona, e sobre ele é possível acoplar um renderizador moderno com widescreen, alta resolução e taxa de quadros alta.

A **decompilação** busca um código que humanos consigam ler e modificar. Decompiladores conhecidos da indústria, como o Ghidra (gratuito, mantido pela NSA) e o IDA, fazem uma primeira tentativa automática, usada também em auditoria de software, interoperabilidade e análise de malware (veja o Livro 4, Capítulo 4, Engenharia Reversa e Segurança).

Os projetos de fãs popularizaram a **decompilação com correspondência** (*matching decompilation*): um código em C que, compilado com o mesmo compilador e as mesmas opções da época, gera um binário **idêntico byte a byte** ao original. Essa exigência funciona como prova objetiva de que a reconstrução está certa. Quando o jogo todo "bate", o grupo tem um código-fonte equivalente ao da empresa, com nomes inventados pelos fãs, e corrigir bugs ou adicionar widescreen vira programação comum.



Espectro das técnicas, da emulação (mais fiel e menos trabalhosa por jogo) ao remaster oficial (mais mudanças e mais questões de direitos). Fonte: elaboração própria.

Você sabia?

Ralph Baer, engenheiro da Sanders Associates, criou a "Brown Box", protótipo do Magnavox Odyssey, primeiro console doméstico, lançado em 1972. O protótipo original está hoje no Smithsonian.

O fenômeno: casos públicos e onde a IA entra

Para começar

Durante décadas, jogar um clássico fora do console original significava emular. Por volta de 2019, voluntários terminaram de reconstruir o código de jogos inteiros do Nintendo 64, e isso abriu caminho para ports e mods. Esta seção percorre os casos mais conhecidos e mostra onde a IA passou a ajudar.

Super Mario 64, Ocarina of Time e Perfect Dark

A decompilação de *Super Mario 64* (1996), concluída em 2019, é o exemplo mais citado de decompilação com correspondência. Dela saiu um port nativo para PC em 2020, e, segundo o Video Games Chronicle, links de download do executável começaram a sumir após pedidos de remoção em nome da Nintendo. A distinção é importante: o repositório de decompilação não continha gráficos nem músicas, enquanto os executáveis prontos distribuídos por terceiros embutiam conteúdo protegido.

Ocarina of Time teve a decompilação concluída no fim de 2021, e em 2022 o grupo Harbour Masters lançou o port **Ship of Harkinian**, que exige que o usuário forneça a própria ROM, obtida legalmente, e trouxe resolução ultralarga, mods e opções que o original nunca teve. *Perfect Dark* foi declarado decompilado em outubro de 2022, com um placar de cerca de 97,4%; segundo o desenvolvedor Ryan Dwyer, o restante eram funções equivalentes que não "batiam" byte a byte.

N64Recomp: recompilar em vez de decompilar

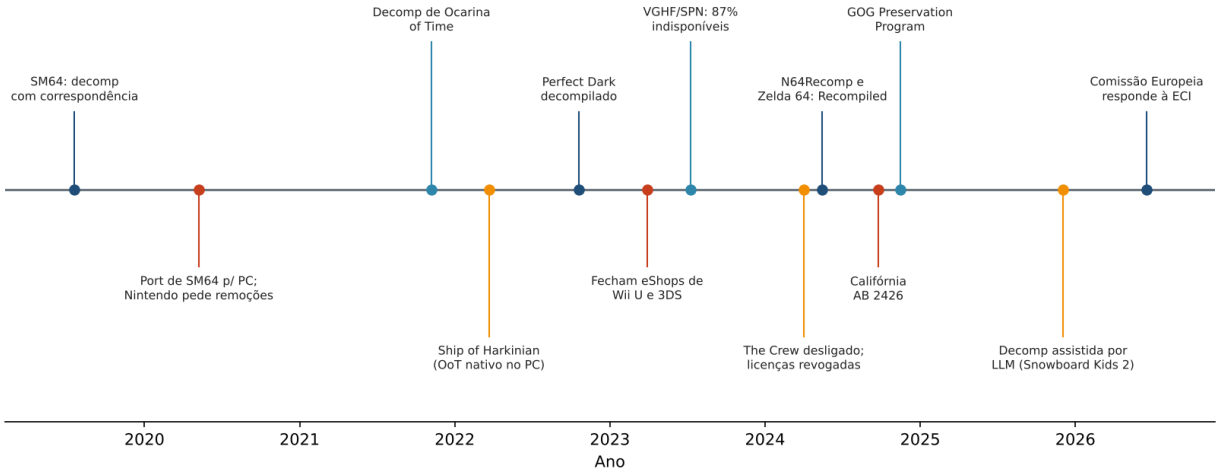
Decompilar com correspondência leva anos por jogo. Em 2024, o **N64Recomp**, de código aberto, passou a recompilar estaticamente binários de N64 para código C, e a primeira vitrine foi o **Zelda 64: Recompiled**, com *Majora's Mask* para Windows e Linux. Como nos outros casos, os assets não são distribuídos. As duas abordagens se complementam: a recompilação dá o executável rápido; a decompilação dá nomes e estrutura para quem quer modificar.

Traduções de fãs

A forma mais antiga de ressignificar um jogo é traduzi-lo. Desde os anos 90, grupos de fãs adaptam textos e fontes (muitos jogos japoneses não tinham "ç" ou "ã") e distribuem o resultado como **patch**, um arquivo só com as diferenças, que o jogador aplica à própria cópia. No Brasil, foi assim que muita gente jogou RPGs inteiros em português pela primeira vez.

Onde a IA entra

- **Decompilação assistida por LLM** (modelo de linguagem, como os chatbots). O caso público mais documentado é o de Chris Lewis com *Snowboard Kids 2* (N64): um modelo propõe o código de cada função, começando pelas mais simples, e o compilador da época confere se os bytes batem. O próprio verificador decide o que conta como progresso. Segundo Simon Willison, o progresso saltou para cerca de 45% no início de dezembro de 2025; depois, o autor relatou que o projeto estacionou perto de 75%, porque o que restava era difícil: funções enormes, rotinas gráficas cheias de macros e funções matemáticas que resistiram por meses.
- **Ferramentas de análise com IA.** Agentes de IA ligados por MCP (um padrão para conectar a IA a outros programas; veja o Livro 2, Capítulo 4, Agentes e MCP) a ferramentas como Ghidra e IDA ajudam a nomear funções e explicar trechos de código. O vídeo A NOVA ONDA DE CLONES DE SOFTWARE | COMO O REA FUNCIONA, do canal Código Fonte TV, lembra que a engenharia reversa sempre existiu; a novidade é a velocidade.
- **Texturas e tradução.** Modelos de super-resolução ampliam texturas antigas, e LLMs traduzem diálogos respeitando contexto e limite de caracteres. Em ambos, os melhores resultados usam a IA como rascunho e um humano para revisar.
- **Criação de jogos.** O vídeo O Jeito CERTO de Criar JOGOS com IA (Poucos Ensinam Assim), do canal Azevedo - Academy, mostra agentes de código e a engine Godot permitindo que iniciantes montem jogos.



Linha do tempo de casos de decompilação, recompilação, desligamentos e reações legais e de mercado (2019–2026). Fonte: elaboração própria com dados das reportagens listadas em "Fontes deste capítulo".

Atenção

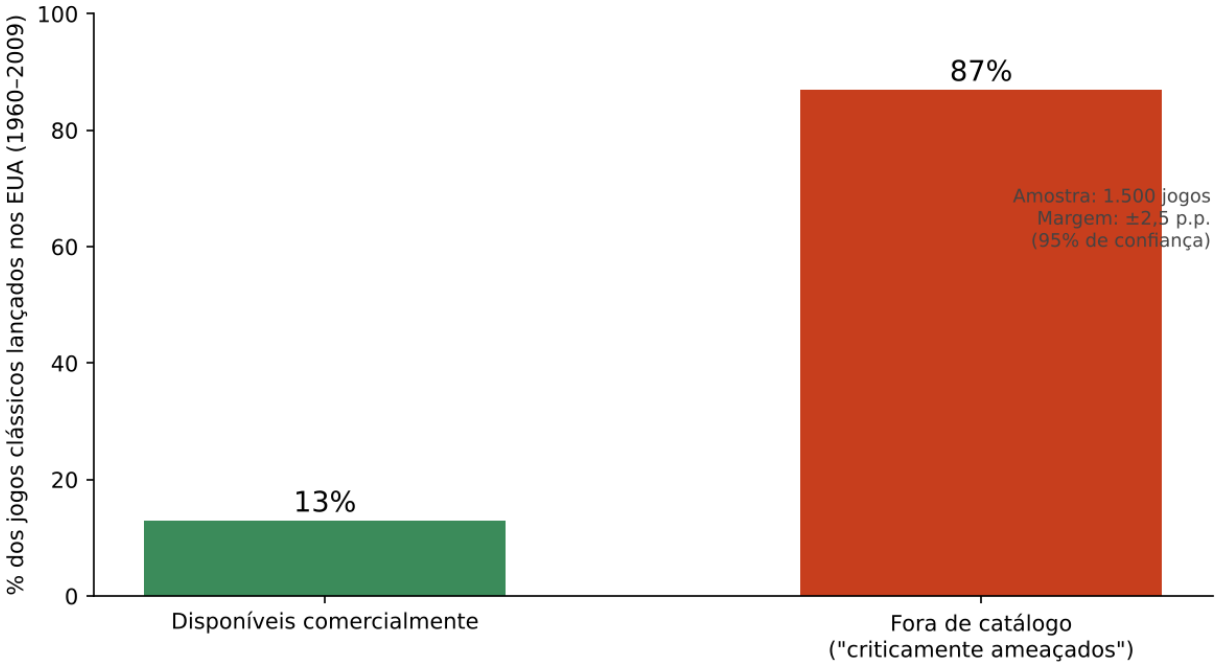
Um erro comum é dizer que "a IA decompila qualquer jogo sozinha". O modelo acelera a parte fácil e média, mas as funções difíceis continuam exigindo especialistas. O que dá confiança ao resultado é o **verificador** (o compilador que confirma os bytes); sem ele, código gerado por IA é só hipótese.

Por que isso acontece: o que os jogadores relatam

Para começar

Por trás de cada port de fã ou tradução há uma frustração concreta: "eu queria jogar isso e não havia como". Às vezes o jogo não está à venda; às vezes está, mas sem português ou só num console caro.

Em 2023, a Video Game History Foundation (VGHF) e a Software Preservation Network sortearam 1.500 jogos lançados nos EUA entre 1960 e 2009 e verificaram quais podiam ser comprados legalmente. Resultado: 87% estão fora de qualquer lançamento comercial, e nenhum período estudado chegou a 20% de disponibilidade. O número mede disponibilidade *comercial*, não perda: muitos desses jogos existem em coleções e no mercado de usados.



Disponibilidade comercial de jogos clássicos lançados nos EUA entre 1960 e 2009 (amostra de 1.500 títulos). Fonte: elaboração própria com dados de Video Game History Foundation e Software Preservation Network, "87% Missing" (2023).

A comparação que o estudo popularizou é com o cinema mudo: os jogos estariam "desaparecendo" do mercado num ritmo comparável ao dos filmes da era silenciosa. O estudo também lembra uma assimetria legal nos EUA: bibliotecas e arquivos podem preservar jogos digitalmente, mas, ao contrário de livros, filmes e áudio, só podem oferecê-los para consulta dentro das próprias instalações.

Causa relatada	Exemplo	Resposta da comunidade
Sem port para a plataforma do jogador	Exclusivos que nunca chegaram ao PC ou ao Steam Deck.	Ports nativos.
Falta de localização em PT-BR	RPGs lançados só em japonês ou inglês.	Patches de tradução, inclusive com IA.
Servidores desligados	*The Crew* (Ubisoft), em 2024.	Campanhas como Stop Killing Games.
Jogos retirados das lojas	eShops do Wii U e do 3DS deixaram de vender em 27/03/2023.	Emulação, mercado de usados.
Hardware que se degrada	A bateria do cartucho acaba e o save some.	Backup pessoal, emulação, consoles recriados em chip (FPGA).

Seria desonesto atribuir todo o fenômeno a motivações nobres. Parte do público que usa emuladores e ROMs simplesmente não quer pagar, inclusive por jogos que estão à venda. Empresas também apontam que relançamentos custam caro (licenças de música e marcas, revisão de código, testes, classificação etária em cada país) e que, para muitos títulos antigos, a demanda não cobre esse custo. E há jogos cujo código-fonte e direitos estão tão fragmentados que um relançamento oficial é inviável mesmo com boa vontade. O ponto do estudo da VGHF é que, nesses casos, a lei hoje impede até que bibliotecas preencham o vazio, e é isso que alimenta a solução "por conta própria".

A degradação do hardware é a causa mais silenciosa. Cartuchos com save dependem de baterias que duram décadas, não para sempre; discos ópticos podem se deteriorar; con-

soles dependem de peças que não são mais fabricadas. Mesmo quem guarda tudo com carinho está numa corrida contra a química.

A lição para as empresas

> **Para começar**
> Quando as pessoas abrem uma trilha no gramado da praça, o paisagista inteligente pavimenta a trilha em vez de pôr uma cerca. Cada port ou tradução de fã funciona como pesquisa de mercado gratuita.

Em 2024, a GOG lançou o **GOG Preservation Program**, que relançou 100 clássicos com o compromisso de mantê-los compatíveis com sistemas modernos: transformou em produto o que os jogadores faziam sozinhos. Outras respostas possíveis são ports oficiais, legendas em português nos relançamentos, modo offline e um **plano de fim de vida** público, que diz na hora da compra o que acontece quando o serviço acabar.

A indústria tem exemplos de sobra de modders que viraram parceiros ou funcionários, de mods que viraram jogos comerciais e de remasters feitos com a ajuda de quem já conhecia o código melhor do que ninguém. O padrão é sempre o mesmo: a empresa detém os direitos e a distribuição; a comunidade traz conhecimento, tempo e entusiasmo. Em termos de risco, um acordo também protege a empresa: o jogo passa a ter um canal oficial, com versão estável, em vez de dezenas de cópias não oficiais.

O argumento a favor de litigar é real: as empresas têm o direito e às vezes o dever de proteger suas marcas, e um port não oficial pode competir com um relançamento futuro. Mas os custos também são reais. Remoções em massa geram cobertura negativa, raramente eliminam os arquivos da internet e transformam um público que queria pagar em um público que não confia mais na marca. O caso *Super Mario 64* ilustra bem: as remoções de 2020 não impediram que o código continuasse circulando, e os ports de fãs seguem existindo anos depois. Já a GOG mostrou que dá para cobrar pelo serviço de "fazer funcionar".

Mídia física e revistas: a memória cultural

> **Para começar**
> Emprestar um cartucho, trocar fitas na locadora e procurar o detonado na revista da banca faziam parte de como o jogo circulava e era lembrado.

Um cartucho ou disco completo funciona sem internet e pode ser emprestado, revendido ou herdado; é dessas cópias que vêm as referências usadas pela preservação. Essa garantia está diminuindo: muitos discos exigem atualizações no primeiro uso, e o Switch 2 trouxe o **Game-Key Card**, cartucho que traz só a autorização para baixar o jogo. Segundo a Nintendo Life, esses cartões não ficam presos a uma conta, o que preserva a revenda, mas o jogo depende de o servidor existir.

No Brasil, antes da internet popular, as revistas eram a principal fonte de informação. A **Ação Games** começou em 1991, e há um acervo digitalizado no Internet Archive com edições de 1991 a 2003; **SuperGamePower**, **EGM Brasil** e **Nintendo World** também marcaram época. Elas registravam preços, lançamentos, locadoras e cartas de leitores: são fontes primárias da história dos games no país.

A memória cultural de um jogo não está só no código. Está no manual, na arte da caixa, no comercial da TV, na revista que explicava como passar do chefe e na lembrança de quem jogou junto. Preservar o executável sem esse contexto é como guardar um filme sem saber quem o assistiu, onde e por quê.

> **Você sabia?**
> A decompilação de *Super Mario 64* foi feita à mão por fãs, função por função, até que o código recompilado gerasse um binário idêntico ao do cartucho. A técnica de "matching decompilation" virou padrão em projetos de preservação.

Propriedade de cópias digitais: "licença, não compra"

> **Para começar**
> Um livro de papel é seu. Um jogo "comprado" numa loja digital, na maioria dos casos, é uma **licença de uso**, como um ingresso de parque: se o parque fecha, o ingresso não vale nada.

The Crew (Ubisoft, 2014) exigia servidor até para jogar sozinho. Os servidores foram desligados em 31 de março de 2024, e jogadores relataram que o jogo foi retirado das bibliotecas digitais. Na França, a associação UFC-Que Choisir processou a Ubisoft com apoio do movimento Stop Killing Games; os processos seguiam em andamento na data de verificação.

O movimento, iniciado por Ross Scott, virou na UE a iniciativa de cidadãos "Stop Destroying Videogames", que reuniu mais de 1,29 milhão de assinaturas válidas e recebeu resposta da Comissão em 16 de junho de 2026. Segundo o GamingOnLinux, a Comissão não propôs nova lei, mas anunciou conversas para um código de conduta sobre o fim de vida dos jogos.

Na Califórnia, a lei **AB 2426**, em vigor desde 2025, determina que lojas de bens digitais revogáveis não usem "comprar" sem deixar claro que se trata de uma licença. A lei não proíbe a revogação; obriga a avisar. O problema vai além dos jogos: em 2023, a Sony anunciou a remoção de conteúdos do Discovery comprados na PlayStation Store e voltou atrás após renegociar o contrato.

Garantia	O que significa na prática	Exemplo / situação atual
Download do instalador	Cópia local que roda sem a loja	GOG (instaladores offline)
Sem DRM ou DRM removível	Jogo abre sem checar servidor	Lojas DRM-free
Modo offline	Partes single-player sobrevivem ao servidor	Patch de fim de vida
Aviso de licença claro	Saber se é compra ou licença	Califórnia AB 2426
Transferência/revenda	Dar ou vender o que pagou	Raro no digital
Herança	Biblioteca passa à família	Quase inexistente
Plano de fim de vida	Data, reembolso, servidor privado	Código de conduta (UE, em discussão)
Reembolso se removido	Dinheiro de volta se perder acesso	Caso a caso

O que uma biblioteca digital deveria garantir ao comprador (instalador offline, rótulo honesto, revenda, herança, plano de fim de vida) e a situação atual de cada garantia. Fonte: elaboração própria a partir dos casos e leis discutidos neste capítulo.

Você sabia?

Nos EUA, a Biblioteca do Congresso concede isenções periódicas à lei DMCA que permitem a bibliotecas e arquivos preservar jogos cujos servidores foram desligados. Mesmo assim, o acesso ao acervo costuma ficar restrito às próprias instalações da instituição.

Ética e legalidade

Para começar

Estudar como uma fechadura funciona, fazer uma chave nova para a própria casa e copiar a chave do vizinho envolvem a mesma técnica, mas só a última é claramente errada. Esta seção traz critérios gerais, não aconselhamento jurídico; e este livro não ensina a obter jogos de forma ilegal.

Atividade	Em geral	Pontos de atenção
Pirataria: espalhar cópias do jogo sem autorização.	Ilegal, inclusive no Brasil.	Jogo "abandonado" (abandonware) não fica liberado.
Preservação: um museu guarda o jogo.	Alguns países permitem.	Guardar não é distribuir.
Engenharia reversa: abrir o programa para entender como funciona.	Permitida em vários países, com condições (por exemplo, na União Europeia).	Contratos podem limitar; quebrar a trava anticópia (DRM) é mais grave.

A distinção entre as três atividades está na finalidade e no que é distribuído, não na técnica: a mesma análise de binários serve a pesquisadores de segurança que estudam malware, a empresas que precisam integrar sistemas legados e a museus que preservam software. Os grandes projetos de fãs distribuem só **código escrito pelos voluntários**, exigem a cópia do jogador e publicam traduções como patches. Esse desenho reduz o risco, mas não torna tudo legal: um código reconstruído para reproduzir o original pode ser visto como obra derivada. Uma régua prática: se há versão oficial acessível, compre; dis-

tribua só o seu trabalho; pergunte quem é prejudicado; e, em projetos grandes, consulte um profissional. A IA acelera a técnica, mas não muda a ética nem a lei.

Fechamento: o jogo como patrimônio

Os jogadores querem jogar o que amam no aparelho que têm, no idioma que falam, sem medo de perder o que pagaram. Quando o mercado não oferece isso, parte da comunidade constrói por conta própria, e a IA torna essa construção mais rápida. A pesquisa da VGHF, o programa da GOG, a lei da Califórnia e a resposta europeia mostram que o tema chegou a reguladores e ao mercado. A pergunta da próxima década é quem vai cuidar dos jogos antigos: empresas, instituições de memória, fãs ou, idealmente, todos juntos.

Fontes deste capítulo

Páginas consultadas em 30/10/2026. Leis, processos e programas comerciais podem mudar depois dessa data.

Vídeos

Título	URL	Autor ou canal	Ponto principal
A NOVA ONDA DE CLONES DE SOFTWARE | COMO O REA FUNCIONA	https://www.youtube.com/watch?v=-soAkLdbNfY	Código Fonte TV	IA acelera a engenharia reversa
O Jeito CERTO de Criar JOGOS com IA (Poucos Ensinam Assim)	https://www.youtube.com/watch?v=fIdQ5mAMOS8	Azevedo - Academy	Criar jogos com agentes e Godot

Páginas web

Título	URL	Autor ou veículo	Ponto principal
87% Missing: the Disappearance of Classic Video Games	https://gamehistory.org/87percent/	Video Game History Foundation	87% dos clássicos fora de catálogo
One-shot decompilation	https://simonwillison.net/2025/Dec/6/one-shot-decompilation/	Simon Willison	LLM acelera decompilação com correspondência
The Long Tail of LLM-Assisted Decompilation	https://blog.chrislewis.au/the-long-tail-of-llm-assisted-decompilation/	Chris Lewis	Progresso estaciona em ~75%
Nintendo takes action against Mario 64 PC port	https://www.videogameschronicle.com/news/nintendo-takes-action-against-mario-64-pc-port/	Video Games Chronicle	Remoções do port em 2020
A full Zelda: Ocarina of Time PC port is now complete and available online	https://www.videogameschronicle.com/news/zelda-ocarina-of-time-pc-port/	Video Games Chronicle	Ship of Harkinian exige ROM do usuário
Perfect Dark has been fully decompiled	https://videogameschronicle.com/news/perfect-dark-has-been-fully-decompiled-making-pc-ports-and-mods-possible	Video Games Chronicle	Decompilação concluída em 2022
Zelda Majora's Mask gets a PC port with a new open source tool	https://gamingonlinux.com/2024/05/zelda-majoras-mask-gets-a-pc-port-with-a-new-open-source-tool-for-nintendo-64-games	GamingOnLinux	N64Recomp e Zelda 64: Recompiled
GOG re-releases 100 improved classic games	https://www.gog.com/pressroom/gog-re-releases-100-improved-classic-games-as-part-of-the-new-gog-preservation-program	GOG	Lançamento do GOG Preservation Program
The Wii U and 3DS eShops close down permanently today	https://www.videogameschronicle.com/news/reminder-the-wii-u-and-3ds-eshops-close-down-permanently-today	Video Games Chronicle	Fim das eShops em 27/03/2023

Título	URL	Autor ou veículo	Ponto principal
Ubisoft removes The Crew from libraries	https://www.gfinityesports.com/article/ubisoft-pulls-the-crew-digital-copies-from-libraries	Gfinity	The Crew retirado das bibliotecas
France's leading consumer association sues Ubisoft over The Crew shutdown	https://www.pcgamer.com/games/frances-leading-consumer-association-backed-by-stop-killing-games-sues-ubisoft-over-the-crew-shutdown	PC Gamer	Ação da UFC-Que Choisir contra Ubisoft
European Commission replies to "Stop Destroying Videogames"	https://citizens-initiative.europa.eu/news/european-commission-replies-stop-destroying-videogames-initiative-2026-06-16_cs	Comissão Europeia	1,29 milhão de assinaturas; resposta oficial
European Commission rejects new laws for Stop Destroying Videogames	https://www.gamingonlinux.com/2026/06/european-commission-rejects-new-laws-for-stop-destroying-videogames/page=1/	GamingOnLinux	Sem nova lei; código de conduta
AB 2426: New California Law Requires Clear Licensing Disclosures for Digital Goods	https://www.gtlaw.com/en/insights/2024/12/ab-2426-new-california-law-requires-clear-licensing-disclosures-for-digital-goods	Greenberg Traurig	Regras para "comprar" bens digitais
PlayStation will no longer remove 1,300 Discovery TV shows	https://www.thefpsreview.com/2023/12/27/playstation-will-no-longer-remove-1300-discovery-tv-shows-from-its-platform-next-month-weve-worked-with-warner-bros-to-update-our-licensing-agreements/	The FPS Review	Remoção de conteúdo comprado revertida
Switch 2 Game-Key Cards aren't tied to Nintendo accounts	https://www.nintendolife.com/news/2025/04/switch-2-game-key-cards-arent-tied-to-nintendo-accounts	Nintendo Life	Cartuchos só com chave de download
Acervo Completo Revistas Ação Games - 1991 / 2003	https://archive.org/details/acaogames147janeiro2000poster/A%C3%A7%C3%A3o%20Games%20%23001%20%28Maio%201991%29/mode/2up	Internet Archive	Acervo digitalizado da Ação Games

CAPÍTULO 3 ·

Uma volta no tempo: dos primeiros jogos à IA nos games

> *Sopre o cartucho (todo mundo fazia isso, mesmo sem funcionar) e aperte Start. Este capítulo é uma homenagem a setenta e poucos anos de jogos eletrônicos, dos pontinhos em osciloscópios aos mundos fotorrealistas de hoje. É também um capítulo de dívida: boa parte da tecnologia que move a IA moderna, a começar pelas GPUs, ficou barata e poderosa porque milhões de pessoas quiseram jogar com gráficos melhores.*

Para começar

Pense na história dos games como a de uma família. Os avós eram cientistas brincando com computadores do tamanho de uma sala; os pais levaram os jogos para bares e salas de estar; os filhos são os estúdios que criaram personagens icônicos; e os netos são as placas de vídeo que hoje treinam modelos de IA.

Os avós: jogos nos computadores de pesquisa

Para começar

Nos anos 1950, ninguém comprava um computador para jogar. Mesmo assim, pesquisadores escreveram jogos neles para demonstrar a máquina, estudar a interação com pessoas e porque era divertido.

Em 1952, Alexander "Sandy" Douglas, aluno de doutorado na Universidade de Cambridge, estudava interação entre humanos e computadores. Para a tese, ele programou o jogo da velha (em inglês, *noughts and crosses*) no **EDSAC** (Electronic Delay Storage Automatic Calculator), um dos primeiros computadores de programa armazenado do mundo, que estava em operação desde 1949. O jogo, chamado **OXO**, mostrava o tabuleiro numa pequena tela de tubo de raios catódicos de 35 por 16 pontos, e o jogador escolhia a casa girando um disco de telefone. O adversário era a própria máquina, que jogava de forma perfeita: no máximo, você empatava.

Em outubro de 1958, o Laboratório Nacional de Brookhaven, em Nova York, abria as portas ao público uma vez por ano. O físico William Higinbotham, que havia trabalhado no Projeto Manhattan, achava as exposições chatas: painéis estáticos e equipamentos que ninguém podia tocar. Ele então montou, com o técnico Robert Dvorak, um jogo de tênis visto de lado num osciloscópio, controlado por um computador analógico. Cada jogador tinha uma caixinha com um botão para rebater e um botão giratório para ajustar o ângulo. A bola caía com gravidade, quicava no chão e batia na rede.

O **Tennis for Two** fez fila. Era, de fato, uma demonstração de física: o computador analógico resolvia equações de trajetória em tempo real, e a "graça" do jogo era ver a matemática funcionando. Higinbotham nunca patenteou a ideia; mais tarde, disse que, se tivesse patenteado, a patente pertenceria ao governo dos EUA, já que ele trabalhava num laboratório federal.



Tennis for Two exibido num osciloscópio DuMont 304-A, em Brookhaven (1958). Foto: Brookhaven National Laboratory, domínio público, via Wikimedia Commons.

Em 1961, a Digital Equipment Corporation (DEC) doou ao MIT um **PDP-1**, um minicomputador "barato" para a época (cerca de 120 mil dólares) e, detalhe crucial, equipado com uma tela de vetores. Um grupo de estudantes e funcionários ligado ao Tech Model Railroad Club, o clube de ferromodelismo que deu origem à cultura hacker, decidiu criar algo que mostrasse o potencial da máquina. Steve Russell escreveu a maior parte do código, com Martin Graetz e Wayne Wiitanen na concepção; Dan Edwards, Peter Samson e Alan Kotok contribuíram com a gravidade da estrela central, o céu estrelado com constelações reais e os controles.

O resultado, **Spacewar!** (1962), punha duas naves (a "agulha" e o "charuto") num duelo em volta de uma estrela que puxava tudo para si. Havia combustível limitado, torpedos li-

mitados e um botão de hiperespaço que podia salvar ou explodir sua nave. Como os controles da máquina eram desconfortáveis, o pessoal improvisou caixas de controle, ancestrais dos joysticks.

O que faz Spacewar! especial não é só ser divertido: ele se espalhou. A DEC passou a incluí-lo como programa de demonstração e teste dos PDP-1 que vendia, e o jogo viajou para dezenas de laboratórios. Em 1972, a revista Rolling Stone organizou as "Olimpíadas Intergalácticas de Spacewar" em Stanford, um dos primeiros torneios de videogame documentados. Dois jovens que jogaram Spacewar! na universidade, Nolan Bushnell e Ted Dabney, tentaram transformá-lo em máquina de fliperama. Daí veio a próxima geração da nossa família.

O padrão se repete: os primeiros jogos foram **demonstrações técnicas**, e jogos são um ótimo teste de estresse, pois exigem tempo real, gráficos, entrada e lógica ao mesmo tempo. Jogos e IA também nasceram juntos: Claude Shannon escreveu sobre xadrez por computador em 1950, e Arthur Samuel criou um programa de damas que aprendia. Depois, Deep Blue (1997) e AlphaGo (2016) mantiveram a tradição de usar jogos como régua da IA.

> **Você sabia?**
> Arthur Samuel popularizou a expressão *machine learning* num artigo de 1959 sobre seu programa de damas, que melhorava jogando contra si mesmo. A mesma ideia de "autojogo" foi levada ao extremo pelo AlphaGo Zero em 2017.

Os pais: o nascimento dos consoles

> **Para começar**
> Um jogo num computador de laboratório é como um fogão industrial: funciona, mas ninguém leva para casa. A virada dos anos 1970 foi fazer o jogo caber na TV da sala e nos fliperamas dos bares.

Ralph Baer, engenheiro nascido na Alemanha que fugiu do nazismo com a família e trabalhava para a fornecedora militar Sanders Associates, teve em 1966 uma ideia simples: os Estados Unidos tinham dezenas de milhões de televisores, e eles só serviam para assistir à programação. E se a TV também servisse para jogar? Com os colegas Bill Harrison e Bill Rusch, ele construiu uma série de protótipos; o mais famoso, a "Brown Box" (caixa marrom, porque era revestida com adesivo imitando madeira), já tinha pingue-pongue, jogos de perseguição e até uma pistola de luz.

A Magnavox licenciou a ideia e lançou, em 1972, o **Magnavox Odyssey**, o primeiro console de videogame doméstico comercial. Ele não tinha processador nem cor: desenhava apenas pontos e uma linha na tela. Para "colorir" os jogos, vinham películas plásticas translúcidas que o jogador grudava na TV por eletricidade estática. Também vinham dados, cartas, fichas e notas de dinheiro de brinquedo, porque parte da graça estava fora da tela. Os "cartuchos" eram placas de circuito que apenas trocavam as ligações internas

do aparelho. Vendeu cerca de 350 mil unidades, e parte do público achava que ele só funcionava em TVs da Magnavox, um equívoco que a propaganda da época ajudou a criar.



O Magnavox Odyssey (1972), o primeiro console doméstico comercial. Foto: Evan-Amos, domínio público, via Wikimedia Commons.

Nolan Bushnell e Ted Dabney já tinham tentado transformar Spacewar! num fliperama, o **Computer Space** (1971), fabricado pela Nutting Associates. Era bonito, com um gabinete de fibra de vidro futurista, mas complicado demais para o frequentador de bar: vendeu pouco. A lição foi aprendida. Em junho de 1972, os dois fundaram a **Atari**, nome tirado de um termo do jogo de tabuleiro Go (algo como "xeque", quando um grupo de pedras está a um lance de ser capturado).

O primeiro engenheiro contratado, Allan Alcorn, recebeu um "exercício de treino": fazer um jogo de pingue-pongue simples. Bushnell tinha visto o jogo de tênis do Odyssey numa demonstração, em maio de 1972. Alcorn caprichou: dividiu a raquete em segmentos, de modo que a bola saía em ângulos diferentes conforme a parte que batia, e acelerou a bola durante a troca de golpes. O resultado ficou tão bom que virou produto: **Pong**, lançado em novembro de 1972.

A história favorita de todo fã é a do protótipo. A Atari instalou uma máquina de teste no bar Andy Capp's Tavern, em Sunnyvale, na Califórnia. Dias depois, o dono ligou reclamando que a máquina tinha quebrado. Alcorn foi lá e descobriu o defeito: o cofre de moedas estava tão cheio que travou o mecanismo. Pong tinha uma instrução só, "evite perder a bola para ter a maior pontuação", e não precisava de mais nada.

Em 1977, a Atari lançou seu console doméstico de cartuchos, o **Atari 2600**. Ele era uma aula de economia de hardware: um processador MOS 6507 (versão barata do 6502) a cerca de 1,19 MHz e apenas **128 bytes de RAM**. Isso mesmo: bytes. Esta frase que você está lendo ocupa mais memória que a RAM do console. Não havia memória de vídeo para guardar uma imagem inteira; o chip gráfico TIA desenhava a tela linha por linha, e o programa precisava "correr atrás do feixe" do televisor, atualizando registradores no mo-

mento exato em que cada linha era desenhada. Programadores chamam isso de *racing the beam*. Para financiar o lançamento, Bushnell vendeu a Atari à Warner Communications em 1976, por cerca de 28 milhões de dólares, e deixou a empresa pouco depois, em 1978.

> **Você sabia?**
> O primeiro "easter egg" famoso nasceu de um protesto: a Atari não creditava seus programadores, e Warren Robinett escondeu seu nome numa sala secreta de Adventure (1980). Insatisfeitos com esse tratamento, programadores saíram da Atari e fundaram a Activision, primeira desenvolvedora independente para consoles.

Entre 1980 e 1982, o mercado norte-americano viveu uma febre. Todo mundo quis fazer console e jogo, inclusive empresas de ração para cachorro (a Purina patrocinou um jogo, Chase the Chuck Wagon). Não havia controle de qualidade nem limite de quem podia publicar jogos para o 2600. Resultado: prateleiras entupidas de jogos ruins, consoles demais competindo (Intellivision, ColecoVision, Odyssey², Atari 5200…) e computadores domésticos baratos, como o Commodore 64, oferecendo "um computador para o seu filho estudar que também roda jogos".

Dois jogos da Atari viraram símbolos do desastre. A versão de **Pac-Man** para o 2600 (1982) vendeu milhões, mas foi muito mal recebida, e a Atari tinha fabricado mais cartuchos do que existiam consoles. E **E.T. the Extra-Terrestrial** (1982) foi desenvolvido por Howard Scott Warshaw em cerca de cinco semanas para chegar às lojas no Natal. Encalhou. Em 1983, a Atari enterrou caminhões de cartuchos e equipamentos num aterro em Alamogordo, no Novo México. Por décadas, isso foi tratado como lenda urbana; em 2014, uma escavação filmada para um documentário desenterrou cartuchos de E.T. e de outros títulos, confirmando a história.

O tamanho da queda é impressionante: a receita do mercado de videogames domésticos nos EUA caiu de cerca de 3,2 bilhões de dólares em 1983 para aproximadamente 100 milhões em 1985, uma perda da ordem de 97%. Lojas passaram a recusar videogames. A lição ficou: sem curadoria e sem qualidade, o mercado se autodestrói. Quem aprendeu melhor essa lição foi uma empresa japonesa de cartas.

A **Nintendo** foi fundada em 23 de setembro de 1889, em Quioto, por Fusajiro Yamauchi, para fabricar **hanafuda**, cartas de baralho japonesas decoradas com flores. Ao longo do século XX, a empresa tentou de tudo: serviço de táxi, uma rede de "hotéis do amor" (segundo relatos populares, nunca bem documentados), arroz instantâneo e brinquedos. O engenheiro Gunpei Yokoi criou, em 1966, a Ultra Hand, uma garra extensível que vendeu milhões, e depois a linha Game & Watch (1980), os portáteis de tela de cristal líquido com um jogo cada. O cruzamento direcional (o "d-pad" em forma de cruz) que Yokoi pôs no Game & Watch de Donkey Kong virou padrão de praticamente todo controle desde então.

Em 1981, um jovem artista chamado Shigeru Miyamoto criou o fliperama **Donkey Kong**, que apresentou o carpinteiro "Jumpman", depois rebatizado **Mario**. Reza a lenda que o nome veio de Mario Segale, dono do armazém que a Nintendo of America alugava nos

EUA e que teria cobrado o aluguel atrasado num momento constrangedor. Em julho de 1983, a Nintendo lançou no Japão o **Family Computer**, ou **Famicom**, com um processador derivado do 6502 e um chip gráfico capaz de rolar a tela e mostrar dezenas de sprites (pequenas figuras móveis).

Levar o Famicom aos EUA, logo depois do crash, era um risco enorme. A Nintendo redesenhou o aparelho para parecer um eletrodoméstico, com o cartucho entrando por uma portinhola frontal, como num videocassete, e o chamou de **Nintendo Entertainment System** (NES). No lançamento em Nova York, em 1985, o pacote vinha com o R.O.B., um robozinho que "jogava" junto: era um cavalo de Troia para convencer as lojas de que aquilo era um brinquedo, não um "videogame". E a Nintendo trouxe a lição do crash na forma de um chip de bloqueio, o **10NES**, e do selo de qualidade: só jogos licenciados e aprovados rodavam no console, e cada editora tinha limite de lançamentos por ano. Com **Super Mario Bros.** (1985) no pacote, o NES vendeu cerca de 62 milhões de unidades no mundo e ressuscitou o mercado.



O Nintendo Entertainment System (NES, 1985 nos EUA), com controle e pistola Zapper. Foto: Evan-Amos, domínio público, via Wikimedia Commons.

A **Sega** deu o troco com o **Mega Drive** (1988 no Japão; **Genesis** nos EUA, 1989), com processador Motorola 68000 de 16 bits. Para enfrentar o Mario, a Sega criou em 1991 um mascote veloz e com atitude: **Sonic the Hedgehog**. A Sega of America, sob Tom Kalinske, apostou numa publicidade agressiva ("Genesis does what Nintendon't", o Genesis faz o que a Nintendo não faz) e popularizou o termo "blast processing" para vender velocidade. A Nintendo respondeu com o Super Nintendo (1990/1991), e a "guerra dos consoles" dos anos 1990 virou um capítulo da cultura pop: no recreio, você era Sega ou Nintendo, sem meio-termo.

No fim dos anos 1980, a Nintendo queria um acessório de CD-ROM para o Super Nintendo e fechou acordo com a Sony. O engenheiro da Sony Ken Kutaragi, que já tinha projetado o chip de som do SNES (o SPC700), liderava o projeto, que se chamava "Play Station".

Na feira CES de junho de 1991, a Sony anunciou o produto. No dia seguinte, a Nintendo subiu ao palco e anunciou que faria o acessório com a **Philips**, sem ter avisado a Sony. O motivo central era contratual: o acordo original dava à Sony muito controle sobre os jogos em CD, e a Nintendo não queria ceder o licenciamento, sua galinha dos ovos de ouro.

Humilhada em público, a Sony hesitou, mas Kutaragi convenceu o presidente Norio Ohga a seguir sozinho. O **PlayStation** foi lançado no Japão em 3 de dezembro de 1994 (setembro de 1995 nos EUA), com CDs baratos de fabricar, gráficos 3D por polígonos e uma política amigável com editoras terceiras. Foi o primeiro console a passar de 100 milhões de unidades vendidas. Em 1993, um protótipo do "Play Station" de SNES foi parar nas mãos de um particular e, em 2020, um exemplar foi leiloado por 360 mil dólares. O acessório da Philips, por sua vez, nunca saiu; o que saiu foram alguns jogos licenciados de Zelda e Mario para o CD-i da Philips, hoje lembrados mais como piada do que como clássicos.

Em 1995, na primeira E3, a Sega anunciou que o Saturn já estava à venda nos EUA por 399 dólares. Em seguida, Steve Race, da Sony, subiu ao palco, disse "299" e saiu. Foi a palestra mais curta e mais eficiente da história das feiras de games.

No fim dos anos 1990, a Microsoft temia que o PlayStation 2 virasse o computador da sala, deixando o Windows de fora. Um grupo de engenheiros da equipe do **DirectX** (a interface de programação de gráficos e som do Windows para jogos), entre eles Seamus Blackley, Kevin Bachus, Otto Berkes e Ted Hase, propôs um console baseado em peças de PC. O nome provisório, "DirectX Box", encurtou para **Xbox**. O aparelho, lançado em 15 de novembro de 2001, tinha processador Intel Pentium III, chip gráfico NVIDIA (o NV2A) e um disco rígido interno, coisa rara em console. A Microsoft chegou a negociar a compra da Nintendo, segundo executivos da época; a proposta teria arrancado risadas na sala de reunião.

No Brasil, a **reserva de mercado de informática** restringia importações e gerou fabricantes locais e clones. O Atari chegou pela Polyvox; clones de NES como o Phantom System e o Top Game dominaram antes da Nintendo oficial; e a **TecToy** lançou o Master System em 1989, traduziu e adaptou jogos (como "Turma da Mônica na Terra dos Monstros") e manteve o console à venda por décadas, um caso mundial de longevidade.



O Sega Master System, lançado no Brasil pela TecToy em 1989 e vendido no país, em várias versões, por décadas. Foto: Evan-Amos, domínio público, via Wikimedia Commons.

Console	Lançamento	Vendas aproximadas	Marca registrada
Magnavox Odyssey	1972	350 mil	Películas coloridas na TV
Atari 2600	1977	30 milhões	128 bytes de RAM
NES / Famicom	1983–1985	62 milhões	Selo de qualidade
Mega Drive	1988	~30 a 35 milhões	Sonic
PlayStation	1994	102 milhões	CD e 3D por polígonos
Xbox	2001	~24 milhões	Disco rígido e Xbox Live

Os filhos: as desenvolvedoras e suas origens improváveis

> **Para começar**
> Console sem jogo é peso de porta. Muitos estúdios gigantes começaram de jeitos improváveis, e vários deixaram contribuições técnicas para toda a computação.

No fim dos anos 1980, John Carmack, John Romero, Tom Hall e Adrian Carmack (sem parentesco) trabalhavam na Softdisk, uma editora de software por assinatura em Shreveport, na Louisiana. Carmack descobriu como fazer rolagem suave de tela no PC, algo que se achava impossível sem hardware dedicado; para demonstrar, o grupo recriou, numa noite, a primeira fase de Super Mario Bros. 3 no PC ("Dangerous Dave in Copyright Infringement", o nome já diz tudo). Mostraram à Nintendo, que educadamente recusou. O grupo então pegava os computadores da empresa nos fins de semana para fazer seus próprios jogos, criou **Commander Keen** (1990) para a editora Apogee e fundou a **id Software** em 1991.

Vieram então três marcos. **Wolfenstein 3D** (1992) popularizou o tiro em primeira pessoa com *ray casting*, uma técnica que lança um raio por coluna da tela para descobrir a que distância está a parede. **Doom** (10 de dezembro de 1993) foi muito além: ambientes com alturas diferentes, iluminação por setores e uma estrutura de dados chamada **BSP** (*binary space partitioning*, partição binária do espaço), que ordena as paredes de antemão para desenhar o mapa rapidamente. Doom foi distribuído como **shareware**: o primeiro episódio era grátis e copiável à vontade, e quem gostava comprava o resto. Também trazia partidas em rede local, o "deathmatch", e uma separação clara entre o **motor** (o código que faz o jogo funcionar) e os **dados** (mapas, sons, texturas, guardados em arquivos WAD). Isso permitiu que fãs criassem fases próprias, os **mods**, um dos berços da cultura de criação por usuários.

Quake (junho de 1996) trouxe um mundo totalmente 3D, com modelos poligonais, e uma versão posterior, o GLQuake, usou aceleração gráfica via OpenGL, tornando-se o motivo número um para alguém comprar uma placa 3dfx (veja a seção sobre GPUs). O QuakeWorld otimizou o jogo pela internet discada com técnicas de predição no cliente que até hoje são base de jogos online. A id também **licenciou seus motores** a outros estúdios: o motor de Quake originou o motor da Valve para Half-Life. E Carmack tinha o hábito de liberar o código-fonte dos motores antigos sob licença GPL: o de Doom saiu em 1997 (e sob GPL em 1999), o de Quake em 1999. É por isso que Doom roda hoje em calculadoras, geladeiras, testes de gravidez com tela e até dentro de outros jogos: "Can it run Doom?" (roda Doom?) virou meme e ritual de passagem para qualquer aparelho com um chip.

- **Blizzard**: virou Blizzard porque o nome Chaos Studios já existia; criou Warcraft, StarCraft e World of Warcraft, e o Battle.net (1996).
- **Valve**: Half-Life (1998) contou a história sem cortes de cena; seus mods geraram Counter-Strike. O **Steam** (2003), mal recebido no início, virou a maior loja de PC, e o Proton leva jogos de Windows ao Linux (Capítulo 1 (Comunidades e Linux)).
- **Square e Enix**: diz a lenda que *Final Fantasy* (1987) seria o último jogo de Sakaguchi; a Enix publicou Dragon Quest. Fundiram-se em 2003.
- **Capcom e Konami**: em Street Fighter II, os **combos** surgiram de um efeito colateral do código; o **Código Konami** foi criado para facilitar testes de Gradius.
- **Rockstar**: GTA nasceu na escocesa DMA Design, e um defeito que deixava a polícia agressiva demais acabou definindo o jogo.
- **Epic Games**: começou com ZZT, de Tim Sweeney; a **Unreal Engine** virou motor licenciado por centenas de estúdios e usado até no cinema.
- **Mojang**: Minecraft (2009), comprado pela Microsoft em 2014, é o jogo mais vendido da história e serve de ambiente de pesquisa para agentes de IA.

> **Dica**
>
> O código liberado de Doom, Quake e Wolfenstein 3D é pequeno, legível e cheio de truques de otimização. É um bom exercício de leitura de código, inclusive com ajuda de IA (Livro 3, Capítulo 1 (Programando com IA)).

Os netos: como os games criaram tecnologia

> **Para começar**
> Pintar sessenta vezes por segundo uma tela de dois milhões de pontos é demais para um único pintor rápido (a CPU), mas não para um exército de milhares de pintores simples trabalhando ao mesmo tempo. Essa é a ideia da **GPU**, criada para jogos. E treinar uma rede neural também é fazer a mesma conta simples milhões de vezes em paralelo.

Até meados dos anos 1990, no PC, quem desenhava os gráficos 3D era a própria CPU, em software. Em 1996, a **3dfx Interactive**, fundada em 1994 por ex-engenheiros da Silicon Graphics, lançou a **Voodoo Graphics**: uma placa só para 3D, que ficava ao lado da placa de vídeo comum, ligada a ela por um cabo externo. Ela fazia em hardware o mapeamento de texturas com filtragem bilinear e o *z-buffer* (que decide qual objeto está na frente), e tinha sua própria interface, a **Glide**. A diferença em jogos como GLQuake e Tomb Raider era de outro planeta: imagens suaves em vez de pixels quadrados, com mais quadros por segundo. A Voodoo2 (1998) permitia ligar duas placas em paralelo (SLI, *scan-line interleave*, com cada placa desenhando linhas alternadas).

A 3dfx dominou por pouco tempo. Errou ao fabricar as próprias placas, atrasou produtos e perdeu espaço para concorrentes que suportavam bem o Direct3D e o OpenGL. No fim de 2000, a NVIDIA comprou a maior parte dos ativos e patentes da 3dfx. Mas a 3dfx deixou uma lição: **hardware especializado para uma tarefa muito paralela pode ser dezenas de vezes mais rápido que um processador genérico**. Guarde essa frase; ela é a base da IA moderna.

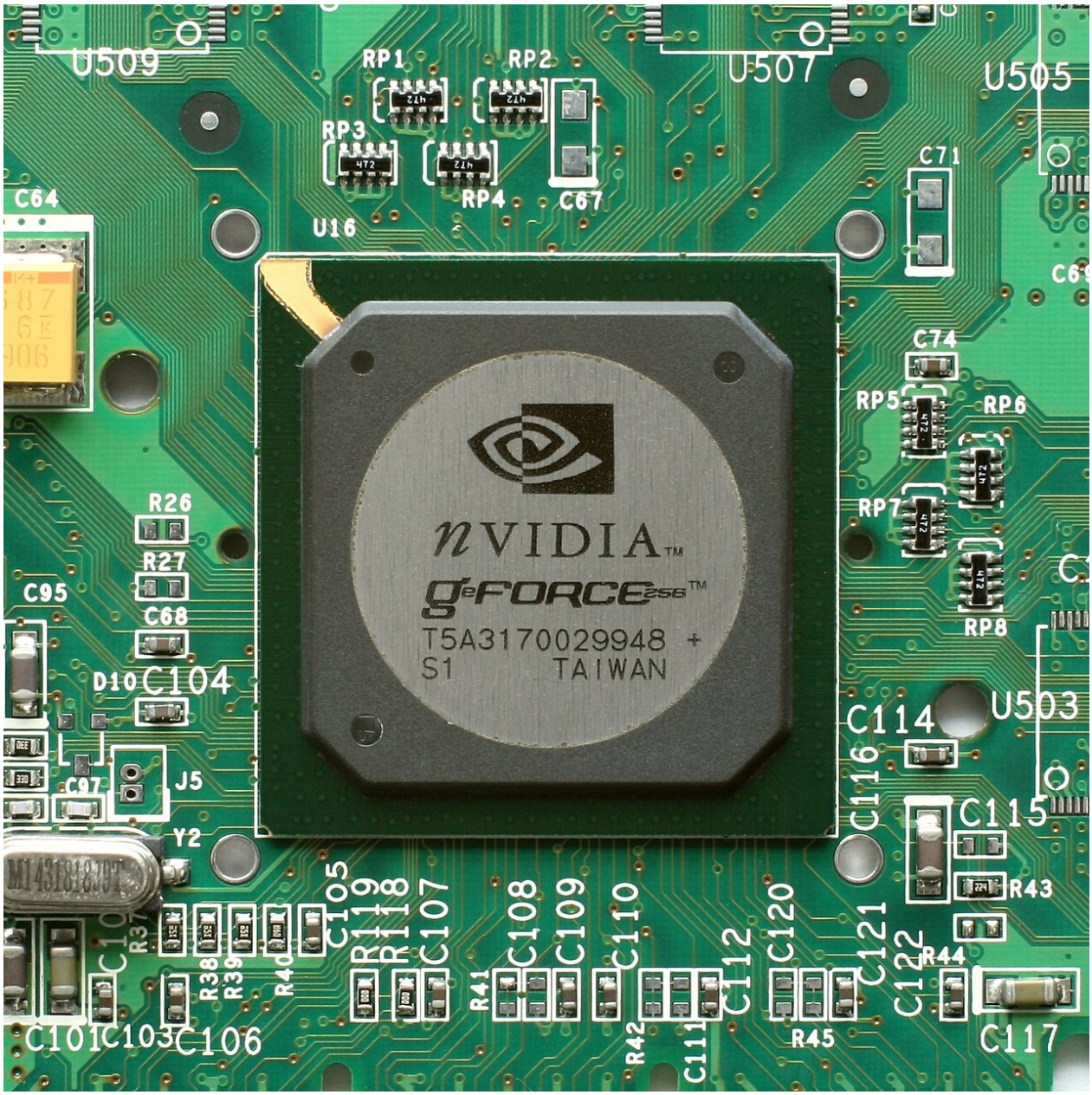
Em 1993, três engenheiros se reuniram numa unidade da rede de lanchonetes **Denny's** em San Jose, na Califórnia, para discutir uma empresa de chips gráficos: **Jensen Huang** (que, na adolescência, tinha trabalhado como lavador de pratos e garçom num Denny's em Portland), **Chris Malachowsky** e **Curtis Priem**, os dois últimos vindos da Sun Microsystems. Fundaram a **NVIDIA** em abril de 1993. Em 2023, a Denny's colocou uma placa comemorativa naquela mesa, chamando-a de "a mesa de 1 trilhão de dólares".

O primeiro produto, o NV1 (1995), apostou em superfícies curvas em vez de triângulos e quase quebrou a empresa, porque a Microsoft padronizou o Direct3D em triângulos. A NVIDIA sobreviveu com a RIVA 128 (1997) e a RIVA TNT (1998). Em 31 de agosto de 1999, anunciou a **GeForce 256**, que o marketing chamou de "a primeira GPU do mundo". A definição da empresa era técnica: um processador num único chip que fazia, em hardware, a transformação e a iluminação dos vértices (**T&L**, *transform and lighting*), além do desenho dos pixels, tirando esse trabalho da CPU. Concorrentes e historiadores discutem o rótulo (outros chips e estações de trabalho já faziam partes disso), mas o termo "GPU" pegou e virou o nome da categoria.

Até 2000, a GPU era uma linha de montagem fixa: fazia sempre as mesmas etapas, com alguns botões para ligar e desligar efeitos. A **GeForce 3** (2001), junto com o DirectX 8, trouxe os **shaders programáveis**: pequenos programas que o desenvolvedor do jogo escreve e que rodam na GPU para cada vértice (vertex shader) ou para cada pixel (pixel

shader). O chip gráfico do Xbox original era, aliás, um parente da GeForce 3. Daí em diante, a linha de montagem virou um exército de processadores programáveis.

A NVIDIA percebeu a oportunidade e, em novembro de 2006, junto com a GeForce 8800 GTX (arquitetura Tesla, com processadores unificados), lançou o **CUDA** (*Compute Unified Device Architecture*): uma plataforma para programar a GPU em uma extensão da linguagem C, sem fingir que tudo era gráfico. A equipe tinha, entre outros, Ian Buck, que havia criado em Stanford a linguagem Brook para GPGPU. A decisão estratégica foi ousada: **toda GeForce**, até as baratas de jogo, passou a rodar CUDA. Isso tornou cada adolescente com um PC gamer dono, sem saber, de um pequeno supercomputador paralelo, e deu a estudantes e pesquisadores um hardware barato para experimentar. Por anos, analistas de Wall Street criticaram o custo do CUDA, que parecia não dar retorno. Em 2012, isso mudou.



Placa NVIDIA GeForce 256 (1999), divulgada como "a primeira GPU". Foto: Konstantin Lanzet, CC BY-SA 3.0, via Wikimedia Commons.

Em 2012, Alex Krizhevsky, Ilya Sutskever e Geoffrey Hinton, da Universidade de Toronto, inscreveram uma rede neural convolucional na competição ImageNet de classificação de imagens. A **AlexNet** foi treinada em **duas placas GeForce GTX 580** (de 3 GB cada), placas de jogo, num computador no quarto de Krizhevsky, na casa dos pais, por cerca de uma semana. Ela reduziu o erro top-5 para cerca de 15,3%, contra 26,2% do segundo colocado. Foi o momento em que a comunidade de visão computacional, e depois toda a área de IA, se convenceu de que **redes profundas + muitos dados + GPUs** funcionavam (Livro 1, Capítulo 1 (Fundamentos de IA)).

A partir daí, a NVIDIA passou a projetar GPUs específicas para data centers (Tesla, depois V100, A100, H100, B200, B300), com núcleos dedicados a multiplicação de matrizes, os **Tensor Cores** (2017, arquitetura Volta). Todos os grandes modelos de linguagem (Livro 1, Capítulo 2 (LLMs e Transformers)) foram treinados nesses descendentes diretos das placas de jogo. Em 2024, a NVIDIA chegou a ser a empresa mais valiosa do mundo em alguns dias de pregão. Jensen Huang costuma dizer que os gamers, com o dinheiro gasto em GeForces durante duas décadas, financiaram a pesquisa que possibilitou a revolução da IA.

> **Você sabia?**
>
> Demis Hassabis, que dividiu o Nobel de Química de 2024 pelo AlphaFold, começou nos games: aos 17 anos foi codesenvolvedor de *Theme Park* (1994). Os primeiros grandes resultados da DeepMind foram agentes que aprendiam a jogar Atari 2600 só olhando os pixels.

Por décadas, jogos usaram **rasterização**: projetar triângulos na tela e pintar os pixels com truques para imitar luz e sombra. O cinema usava **ray tracing** (traçado de raios): simular raios de luz quicando pela cena, o que dá reflexos e sombras realistas, mas era lento demais para tempo real. Em agosto de 2018, a NVIDIA lançou a linha **GeForce RTX 20** (arquitetura Turing), com núcleos dedicados ao cálculo de interseção de raios (**RT Cores**) e com Tensor Cores na placa de jogo. Ray tracing em tempo real virou realidade, ainda que, no início, a um custo alto de desempenho.

Para pagar essa conta, a NVIDIA usou IA: o **DLSS** (*Deep Learning Super Sampling*, lançado em 2019) renderiza o jogo em resolução menor e usa uma rede neural, rodando nos Tensor Cores, para reconstruir a imagem em resolução maior. O DLSS 2 (2020) melhorou muito a qualidade usando informações de quadros anteriores e vetores de movimento. O **DLSS 3** (2022, linha RTX 40) acrescentou a **geração de quadros** (*frame generation*): a rede cria quadros inteiros intermediários entre dois quadros renderizados. O **DLSS 4** (2025, linha RTX 50) trouxe a geração de múltiplos quadros (até três quadros gerados para cada um renderizado) e modelos do tipo transformer (Livro 1, Capítulo 2 (LLMs e Transformers)) para a reconstrução. AMD (FSR) e Intel (XeSS, presente nas placas Arc) oferecem tecnologias concorrentes. Ou seja: a placa de jogo que treinou a IA agora usa IA para desenhar o próprio jogo. O círculo se fechou.

Atenção

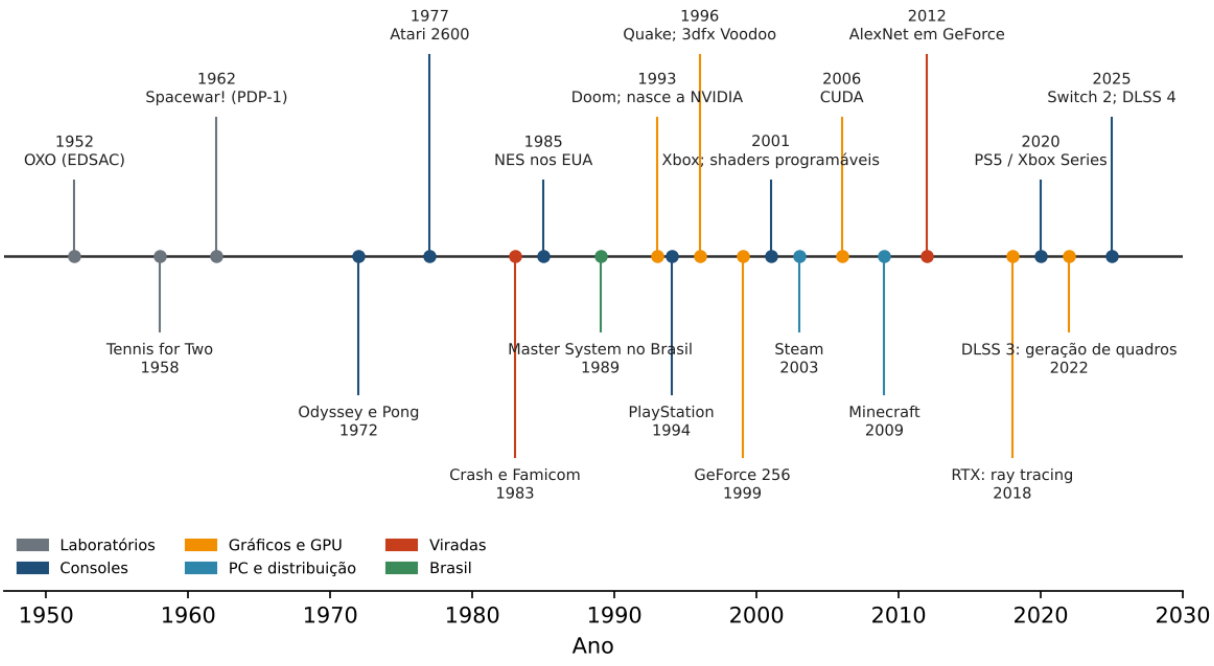
Geração de quadros aumenta os quadros exibidos, mas não reduz a latência de entrada como quadros renderizados, porque os quadros gerados não refletem novos comandos do jogador. Em jogos competitivos, isso importa.

As GPUs não são a única dívida. Motores de física dos games hoje treinam robôs em simulação; jogos online popularizaram predição no cliente e compensação de atraso; a Unreal Engine é usada em produção virtual no cinema (como em *The Mandalorian*) e em simuladores de direção autônoma; e o Kinect levou sensores de profundidade baratos a pesquisadores de robótica. Em resumo, durante mais de uma década os gamers financiaram o hardware paralelo que a IA encontrou pronto quando precisou dele.

Dos clássicos aos lançamentos atuais

Para começar

Dos pontos num osciloscópio a mundos com luz realista passaram-se cerca de setenta anos. Cada era trocou a técnica de desenho: vetores, sprites, polígonos, shaders e, agora, reconstrução por IA.



Linha do tempo dos games e da tecnologia gráfica, de 1952 a 2025, com os marcos citados neste capítulo. Fonte: elaboração própria com base nas fontes listadas ao fim do capítulo.

Console	Ano	RAM principal	Mídia típica
Atari 2600	1977	128 bytes	Cartucho de 2 a 4 KB
NES	1985	2 KB	Cartucho de até ~512 KB
PlayStation	1994	2 MB	CD de 650 MB
PlayStation 5	2020	16 GB	SSD e download de 50 a 150 GB

A RAM cresceu da ordem de cem milhões de vezes em cerca de 45 anos. Ainda assim, os jogos que marcaram gerações têm regras claras e boa sensação de controle: Pong, Tetris, Minecraft, e indies feitos por uma pessoa só, como Stardew Valley e Balatro. Tecnologia ajuda, mas não substitui um bom desenho de jogo, e gerar mais conteúdo com IA não é o mesmo que gerar diversão.

Por que a IA será cada vez mais comum nos games

> **Para começar**
> Por décadas, a "IA do inimigo" foi feita de regras escritas à mão, como o boneco de um parque que repete os mesmos movimentos. Redes neurais e modelos de linguagem permitem personagens que conversam, conteúdo criado na hora e produção acelerada. O uso da IA na preservação de jogos antigos está no Capítulo 2 (IA e games: preservação).

Vale separar duas coisas que usam o mesmo nome. A **IA clássica de jogos** é feita de técnicas determinísticas e baratas: máquinas de estados finitos (os fantasmas de Pac-Man têm personalidades programadas: Blinky persegue direto, Pinky tenta cercar à frente do jogador), algoritmos de busca de caminho como o A* (A-estrela), árvores de comportamento (Halo 2, 2004, popularizou a técnica) e planejadores como o GOAP, usado em F.E.A.R. (2005) para que soldados inimigos parecessem coordenados. Essas técnicas continuam ótimas, porque são previsíveis, rápidas e fáceis de ajustar. Um inimigo "inteligente demais" é frustrante, e o objetivo da IA de jogo é ser divertida, não vencer.

NPCs que conversam

Um **NPC** (*non-player character*, personagem não jogável) costumava ter uma árvore de diálogo com frases fixas. Com um modelo de linguagem (LLM), o NPC pode responder ao que o jogador digitar ou falar, lembrar de conversas anteriores e reagir ao estado do mundo. A NVIDIA apresentou em 2023 a plataforma **ACE** (Avatar Cloud Engine), que combina reconhecimento de fala, LLM, síntese de voz e animação facial (Audio2Face); empresas como a Inworld AI e a Convai oferecem ferramentas de personagens; a Ubisoft mostrou em 2024 um protótipo chamado NEO NPC; e jogos de 2025 começaram a usar personagens com modelos locais pequenos, rodando na GPU do jogador.

A parte técnica é a da engenharia de contexto e dos agentes (Livro 2, Capítulos 2 e 4): o NPC recebe um *system prompt* (as instruções iniciais do modelo) com sua personalidade e regras do mundo, uma memória resumida, e ferramentas para agir no jogo (dar um item, abrir uma porta). Os desafios também: alucinação (o ferreiro inventar uma missão que não existe), quebra de personagem, conteúdo impróprio, latência e custo. Por isso,

os projetos mais sólidos limitam o LLM: ele escolhe entre ações permitidas e preenche o texto, mas a lógica do jogo continua controlada por código.

Geração procedural e modelos generativos

Gerar conteúdo por algoritmo é antigo. **Rogue** (1980) criava masmorras novas a cada partida; **Elite** (1984) cabia num computador de 32 KB com oito galáxias de 256 planetas cada, gerados a partir de números-semente; Minecraft gera mundos com ruído de Perlin; **No Man's Sky** (2016) anuncia 18 quintilhões de planetas. Essa geração procedural clássica usa regras e números pseudoaleatórios, não IA no sentido moderno.

A novidade são os **modelos generativos** criando texturas, modelos 3D, sons, músicas e até o próprio jogo. Em 2024, o GameNGen, do Google, mostrou uma rede neural de difusão gerando quadros de Doom jogável em tempo real, sem motor de jogo tradicional, só prevendo o próximo quadro a partir do anterior e do comando do jogador. A Microsoft mostrou o modelo Muse (2025), treinado em partidas do jogo Bleeding Edge, e o Google DeepMind apresentou os modelos Genie, que geram ambientes interativos a partir de uma descrição. Por enquanto, são demonstrações de pesquisa, com pouca memória de longo prazo e consistência limitada, mas apontam para uma direção: **o modelo como motor de jogo**. Repare que Doom, de novo, serve como régua.

Produção e IA local

A IA moderna também entra na produção:

- **Produção**: animação mais natural, bots que testam fases em busca de falhas, assistentes de código nos motores e tradução assistida com revisão humana.
- **IA local**: modelos pequenos rodando na GPU do jogador evitam custo de servidor e funcionam offline (Livro 3, Capítulo 2 (Hardware e IA local)).

As polêmicas

Há também conflitos reais. O sindicato SAG-AFTRA fez greve contra grandes empresas de jogos a partir de julho de 2024, principalmente por proteções contra réplicas de voz por IA; o acordo ratificado em 2025 exige consentimento e remuneração para réplicas digitais. O Steam passou a exigir, em 2024, que desenvolvedores declarem o uso de IA generativa. E as demissões de 2023 a 2025, embora com causas sobretudo econômicas, aumentaram o medo da categoria. O debate é sobre **quem decide**, **quem é pago** e **quem recebe crédito** (Livro 5, Capítulo 3 (Propriedade intelectual)).

> **Dica**
> Ao usar IA num jogo: declare o uso; não clone vozes ou estilos sem contrato e consentimento; use a IA no que é repetitivo ou provisório; e teste NPCs com LLM contra tentativas de "quebrar" o personagem.

A história dos games é a de gente brincando com máquinas que não foram feitas para brincar e, ao brincar, empurrando-as para a frente. Se a tradição se mantiver, a próxima

grande inovação pode estar sendo testada agora por alguém tentando rodar Doom onde ele não deveria rodar.

Fontes deste capítulo

Dados verificados em 31/10/2026. Imagens: Wikimedia Commons, com autor e licença indicados em cada legenda.

Título	URL	Canal / autor	Ponto principal
OXO (video game)	https://en.wikipedia.org/wiki/OXO_(video_game)	Wikipedia	Jogo da velha no EDSAC, 1952
The First Video Game?	https://www.bnl.gov/about/history/firstvideo.php	Brookhaven National Laboratory	Tennis for Two, 1958
Spacewar! — PDP-1 Restoration Project	https://www.computerhistory.org/pdp-1/spacewar/	Computer History Museum	Spacewar! no PDP-1, 1962
Ralph Baer Papers and Video Game Collection	https://americanhistory.si.edu/collections/object-groups/ralph-baer-papers-and-video-game-collection	Smithsonian	Baer, Brown Box e Odyssey
Pong	https://en.wikipedia.org/wiki/Pong	Wikipedia	Atari, Alcorn e o protótipo
Atari 2600	https://en.wikipedia.org/wiki/Atari_2600	Wikipedia	128 bytes de RAM e vendas
Video game crash of 1983	https://en.wikipedia.org/wiki/Video_game_crash_of_1983	Wikipedia	Causas, queda de receita, Alamogordo
Nintendo History	https://www.nintendo.co.jp/corporate/en/history/index.html	Nintendo	Hanafuda, Game & Watch, Famicom
Nintendo Entertainment System	https://en.wikipedia.org/wiki/Nintendo_Entertainment_System	Wikipedia	Lançamento nos EUA e chip 10-NES
Super NES CD-ROM	https://en.wikipedia.org/wiki/Super_NES_CD-ROM	Wikipedia	Origem do PlayStation
Xbox (console)	https://en.wikipedia.org/wiki/Xbox_(console)	Wikipedia	Origem na equipe DirectX
Tectoy	https://pt.wikipedia.org/wiki/Tectoy	Wikipédia	Master System e Mega Drive no Brasil
id Software	https://en.wikipedia.org/wiki/Id_Software	Wikipedia	Wolfenstein 3D, Doom e Quake
id-Software/DOOM	https://github.com/id-Software/DOOM	id Software (GitHub)	Código-fonte do motor de Doom
Valve Corporation	https://en.wikipedia.org/wiki/Valve_Corporation	Wikipedia	Half-Life, Steam e Steam Deck
3dfx Interactive	https://en.wikipedia.org/wiki/3dfx_Interactive	Wikipedia	Voodoo Graphics e venda à NVIDIA
NVIDIA Corporate Timeline	https://www.nvidia.com/en-us/about-nvidia/corporate-timeline/	NVIDIA	GeForce 256, CUDA, RTX
ImageNet Classification with Deep Convolutional Neural Networks	https://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks	Krizhevsky, Sutskever e Hinton (NeurIPS 2012)	AlexNet treinada em duas GTX 580
NVIDIA DLSS	https://www.nvidia.com/en-us/geforce/technologies/dlss/	NVIDIA	Reconstrução e geração de quadros
The Nobel Prize in Physics 2024	https://www.nobelprize.org/prizes/physics/2024/summary/	Nobel Prize	Hopfield e Hinton
Diffusion Models Are Real-Time Game Engines	https://arxiv.org/abs/2408.14837	Valevski et al. (Google)	GameNGen: Doom gerado por difusão
The videogame voice actors' strike officially ends	https://www.pcgamer.com/gaming-industry/the-videogame-voice-actors-strike-officially-ends-as-performers-ratify-agreement-between-sag-aftra-and-publishers	PC Gamer	Fim da greve da SAG-AFTRA
SAG-AFTRA video game actors approve new agreement	https://www.windowscentral.com/gaming/sag-aftra-approves-new-contract-july-2025	Windows Central	Consentimento e pagamento por réplicas
Wikimedia Commons	https://commons.wikimedia.org/	Wikimedia Foundation	Fotografias com licença livre

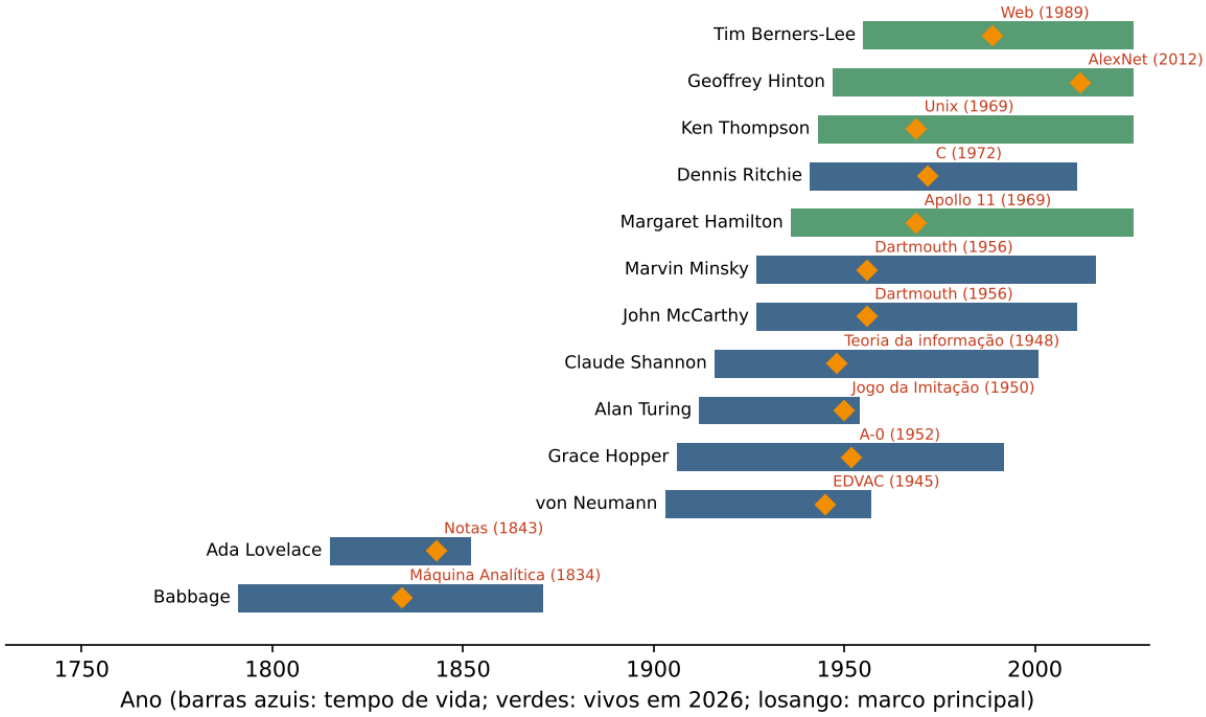
CAPÍTULO 4 ·

E se Ada Lovelace e Alan Turing vissem a computação hoje?

> *Este último capítulo faz um exercício de imaginação: trazer para 2026 as pessoas que criaram as ideias sobre as quais este livro foi construído. Para cada pioneiro separamos três camadas: o que de fato fez (fatos verificados), o que previu ou temeu (com citações curtas dos próprios textos) e, sempre marcado como especulação, como reagiria ao mundo de hoje. Ninguém pode falar por quem já morreu; a obra deles serve aqui como lente para olhar o presente.*

Para começar

Pense em um avô que construiu uma casa e, décadas depois, encontra no lugar um bairro inteiro. Algumas coisas ele reconheceria na hora; outras o deixariam boquiaberto ou preocupado. A computação é essa casa, e os trechos marcados como "especulação" são palpites informados, nunca fatos.

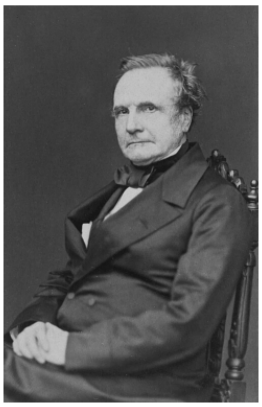


Linha do tempo dos pioneiros deste capítulo: tempo de vida de cada um (verde para quem está vivo em outubro de 2026) e o marco principal associado a seu nome. Fonte: elaboração própria a partir das biografias citadas nas fontes do capítulo.

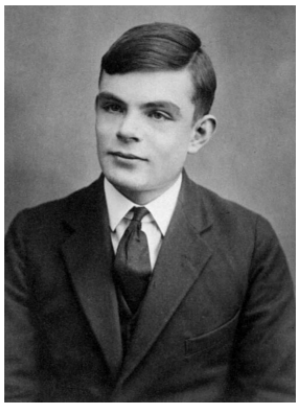
Ada Lovelace
(1815–1852)



Charles Babbage
(1791–1871)



Alan Turing
(1912–1954), aos 16 anos



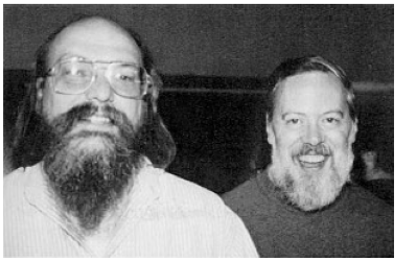
Grace Hopper
(1906–1992)



Margaret Hamilton
(n. 1936)



Ken Thompson e
Dennis Ritchie, 1973



Alguns dos pioneiros: Ada Lovelace (retrato de Alfred Edward Chalon, c. 1840), Charles Babbage (1860), Alan Turing aos 16 anos, Grace Hopper (Marinha dos EUA), Margaret Hamilton (NASA) e Ken Thompson com Dennis Ritchie (1973). Imagens em domínio público, segundo o Wikimedia Commons.

Ada Lovelace e Charles Babbage: a máquina que manipula símbolos

> **Para começar**
> Babbage projetou um computador de engrenagens antes que houvesse eletricidade em casa. Ada Lovelace percebeu que, se números pudessem representar notas musicais ou letras, essa máquina poderia trabalhar com qualquer coisa: a semente do computador de uso geral.

Charles Babbage (1791–1871) projetou a **Máquina Analítica** a partir de 1834, com um "armazém" (memória), um "moinho" (processador), cartões perfurados e desvio condicional. Ela nunca foi construída. Em 1843, Ada Lovelace (1815–1852) traduziu um artigo de Luigi Menabrea sobre a máquina e acrescentou sete notas, quase três vezes maiores que o original. A Nota G descreve como calcular os números de Bernoulli, e por isso Ada é chamada de "a primeira programadora"; historiadores discutem a autoria do algoritmo, mas a visão conceitual das Notas é dela.

Ada previu que a máquina poderia "compor peças musicais elaboradas e científicas de qualquer grau de complexidade". Mas também escreveu: "*The Analytical Engine has no pretensions whatever to originate anything. It can do whatever we know how to order it to perform.*" Turing chamaria isso de "Objeção de Lady Lovelace". Diante de um LLM (modelo de linguagem), a objeção ainda divide opiniões.

- **A favor de Ada:** ninguém "ordenou" ao modelo cada palavra, mas tudo o que ele faz vem de um procedimento definido por pessoas (a arquitetura Transformer, o treino para prever o próximo pedaço de texto, o ajuste com preferências humanas) aplicado a um enorme volume de texto escrito por pessoas. Nesse sentido, ele faz "o que soubemos ordenar", só que a ordem foi dada em nível estatístico, não linha por linha.
- **Contra uma leitura literal:** o próprio Turing respondeu, em 1950, que máquinas o surpreendiam com frequência, e que a ideia de que nada de novo sai delas pressupõe que conseguimos prever todas as consequências do que programamos. Com bilhões de parâmetros, ninguém consegue. Pesquisadores falam em "capacidades emergentes", comportamentos que não foram projetados explicitamente.

Uma leitura mais generosa é que Ada estava certa sobre a máquina de 1843 e fez a pergunta certa para as máquinas de 2026. A questão deixou de ser "a máquina pode fazer algo que não mandamos?" e passou a ser "quem é responsável pelo que ela faz quando não sabemos exatamente o que mandamos?". É a pergunta que atravessa os capítulos sobre agentes (Livro 2), propriedade intelectual e polêmicas da IA (Livro 5).

Babbage deixou outra anedota famosa: duas vezes lhe perguntaram se, colocando números errados na máquina, sairiam respostas certas. É a versão do século XIX de "lixo entra, lixo sai".

> **Você sabia?**
> A Máquina Diferencial nº 2 de Babbage foi montada pelo Science Museum de Londres a partir dos desenhos originais e concluída em 1991, provando que o projeto funcionava. A linguagem Ada, padronizada em 1983, tem o número de padrão militar MIL-STD-1815, o ano de nascimento de Ada Lovelace.

Especulação. Imagine Ada em um estúdio caseiro. Ela abriria um gerador de música por IA, pediria "uma fuga a quatro vozes no estilo de Bach, mas sobre um tema folclórico inglês", ouviria o resultado e logo faria a pergunta que fazia em 1843: quais são as *relações* que a máquina está manipulando? Ela não se contentaria com o áudio. Iria querer a partitura simbólica, o MIDI, as regras de contraponto que o sistema respeitou ou quebrou. Talvez escrevesse uma nova "Nota H" sobre a diferença entre um modelo que gera ondas sonoras diretamente e um que manipula símbolos musicais, e argumentasse que o segundo é mais próximo da sua Máquina Analítica, porque o resultado pode ser lido, criticado e editado.

Também é plausível que ela se incomodasse com a falta de crédito aos artistas cujo trabalho treina os modelos (Livro 5, Capítulo 3 (Propriedade intelectual)), ela que assinou as Notas só com iniciais.

Especulação. A pergunta dos "números errados" que irritou Babbage virou, em 2026, pesquisa séria. Modelos de linguagem às vezes *corrigem* erros da entrada: se você escreve um enunciado com um erro de digitação, o modelo frequentemente entende o que você quis dizer. E às vezes fazem o contrário: recebem dados corretos e devolvem uma "alucinação" confiante. Babbage, que passou a vida caçando erros em tabelas, provavelmente se tornaria um entusiasta de **avaliações** (os *evals* e benchmarks discutidos no Livro 1, Capítulo 2 (LLMs e Transformers) e no Livro 3, Capítulo 1 (Programando com IA)). Dá para imaginá-lo montando uma bateria de testes com resultados conhecidos e publicando, com seu temperamento briguento, um panfleto contra empresas que não divulgam a taxa de erro dos modelos.

Alan Turing: a pergunta que ainda fazemos

> **Para começar**
> Em 1936, Turing provou que há perguntas que nenhuma máquina responde, e para isso inventou no papel o computador moderno. Em 1950, propôs um jogo: se você conversa por escrito com uma máquina e não a distingue de uma pessoa, talvez não faça sentido negar que ela pensa.

Alan Turing (1912–1954) descreveu em 1936 uma máquina abstrata e mostrou que uma "máquina universal" poderia simular qualquer outra, além de provar a indecidibilidade do problema da parada (Livro 4, Capítulo 2 (Algoritmos)). Na guerra, teve papel central na quebra da Enigma em Bletchley Park. Em 1952, foi condenado por ter uma relação homossexual e submetido a castração química; morreu em 1954. Recebeu perdão real póstumo em 2013.

Em "Computing Machinery and Intelligence" (1950), trocou a pergunta "Can machines think?" pelo **Jogo da Imitação** e propôs: em vez de programar uma mente adulta, por que não programar uma mente de criança e educá-la? É, em essência, o aprendizado de máquina. O artigo termina com: "*We can only see a short distance ahead, but we can see plenty there that needs to be done.*" Em 2025, um estudo da UC San Diego mostrou o GPT-4.5, com uma persona, apontado como humano em 73% das vezes num teste de três participantes. O teste durava cinco minutos e mede imitação de conversa, não necessariamente pensamento.

Seu último grande trabalho foi biológico: em 1952 propôs que manchas e listras de animais surgem de substâncias que reagem e se espalham em velocidades diferentes, os "padrões de Turing", depois observados experimentalmente.

> **Você sabia?**
> O Prêmio Turing, entregue pela ACM desde 1966, é chamado de "Nobel da computação". Geoffrey Hinton o recebeu em 2018 e, em 2024, ganhou também o Nobel de Física, sendo uma das poucas pessoas com os dois prêmios.

Especulação. É provável que Turing ficasse menos impressionado com o chatbot que conversa e mais com o AlphaFold, o sistema da DeepMind que previu a estrutura de cen-

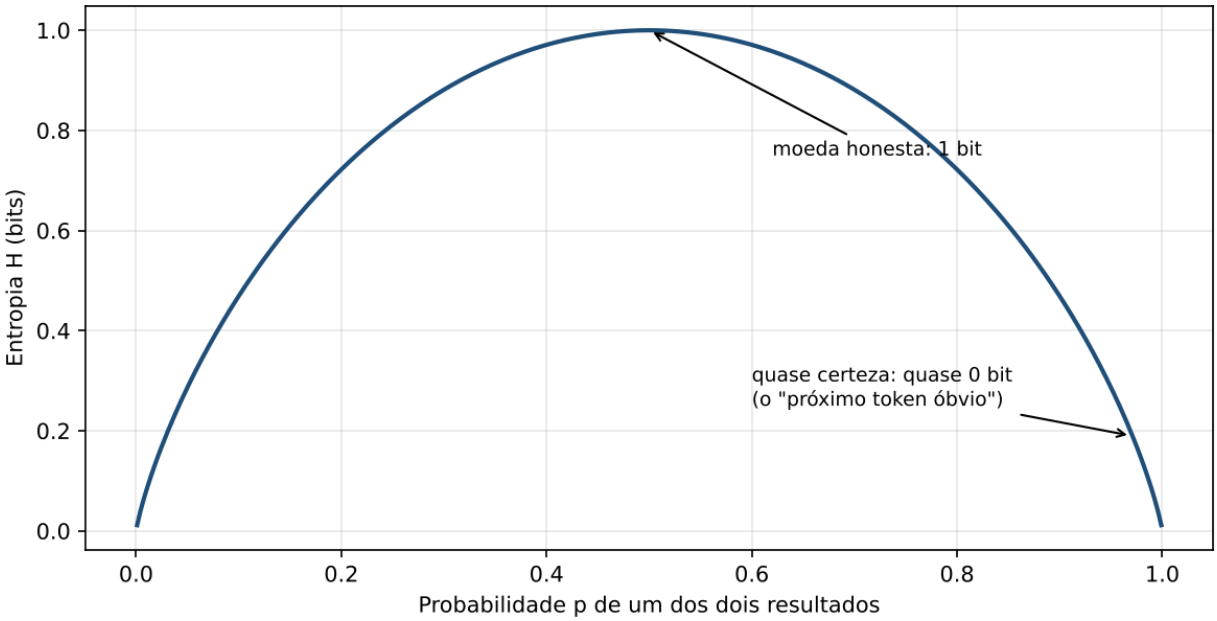
tenas de milhões de proteínas e rendeu metade do Nobel de Química de 2024 a Demis Hassabis e John Jumper. Ele uniria as duas paixões, máquinas que aprendem e biologia que gera forma. Imagine-o pedindo a um agente de programação (Livro 3, Capítulo 1 (Programando com IA)) que escreva uma simulação de reação-difusão na GPU, ajustando parâmetros até reproduzir as manchas de um animal específico, e depois pedindo a um modelo de proteínas que sugira quais moléculas reais poderiam fazer o papel de "ativador" e "inibidor".

John von Neumann e Claude Shannon: arquitetura e informação

> **Para começar**
>
> Todo computador comum guarda na mesma memória os dados e as instruções. E a informação de uma mensagem depende de quanto ela surpreende: saber que o sol vai nascer não ensina nada; saber o número da loteria ensina muito. Essas duas ideias sustentam a computação e, de modo direto, os modelos de linguagem.

John von Neumann (1903–1957) descreveu em 1945, no "First Draft of a Report on the EDVAC", a arquitetura de programa armazenado, ideia também discutida por Eckert e Mauchly. Também criou a teoria dos jogos e, segundo relato de Ulam, falou de uma "singularidade" no progresso tecnológico. **Especulação:** ele se interessaria pelo "gargalo de von Neumann", pois nos LLMs a velocidade de geração muitas vezes depende mais da banda de memória que do cálculo (Livro 3, Capítulo 2 (Hardware e IA local)).

Claude Shannon (1916–2001) mostrou em 1937 que circuitos de relés implementam a álgebra de Boole e, em 1948, criou a teoria da informação, com o "bit" como unidade. No mesmo artigo, gerou textos em inglês com estatísticas cada vez mais ricas: quanto mais contexto, mais o texto parecia inglês. Um LLM é, simplificando, essa ideia com milhares de tokens de contexto. Em 1951, estimou que o inglês tem entre 0,6 e 1,3 bit por letra, o mesmo tipo de medida da **perplexidade** usada hoje para avaliar modelos.



Entropia de um evento com dois resultados, $H(p) = -p \cdot \log_2 p - (1-p) \cdot \log_2(1-p)$. Uma moeda honesta carrega 1 bit; um resultado quase certo, quase nada. Quando um LLM tem certeza do próximo token, a entropia é baixa. Fonte: elaboração própria a partir da definição de Shannon (1948).

A "temperatura" na geração de texto (Livro 2, Capítulo 2 (Engenharia de Contexto)) aumenta ou diminui essa entropia, e comprimir bem é prever bem: LLMs são excelentes compressores de texto.

Especulação. Shannon, que andava de monociclo pela Bell Labs e construiu um rato mecânico que aprendia labirintos, provavelmente passaria um fim de semana medindo a entropia por token de diferentes modelos em textos em português, comparando com a estimativa que ele próprio fez para o inglês, e construindo algum dispositivo absurdo e genial: talvez um robô de malabarismo controlado por um modelo de visão. Ele também se interessaria pelos tokenizadores. Um tokenizador que divide palavras em pedaços frequentes é uma forma de codificação eficiente, parente das ideias de compressão que ele ajudou a criar (a codificação de Shannon-Fano). Dá para imaginá-lo notando que línguas com menos dados de treino, como o português, costumam gastar mais tokens por palavra que o inglês, e que isso tem custo real para quem paga por token.

Você sabia?

Como os tokenizadores foram otimizados principalmente para o inglês, o mesmo texto em português costuma virar mais tokens, e APIs cobram por token. Shannon descreveria isso como uma diferença de eficiência de código.

Grace Hopper: falar com a máquina em inglês

Para começar

Grace Hopper achava absurdo que pessoas aprendessem a língua da máquina, se a máquina podia aprender a língua das pessoas. Seus tradutores, os compiladores, são avós diretos de pedir código a uma IA em linguagem natural.

Grace Hopper (1906–1992) criou em 1952 o A-0, um dos primeiros compiladores, e o FLOW-MATIC, base do COBOL, que ainda roda em bancos e governos. Popularizou a história do "bug" (a mariposa de 1947 no Harvard Mark II) e dizia que a frase mais perigosa é "sempre fizemos assim". **Especulação:** ela veria os assistentes de código como o fim da estrada que abriu, mas notaria que o compilador é determinístico e o LLM não, então o programador passa de escritor a revisor (Livro 3, Capítulo 3 (AI-Driven Engineer)).

Seu lema "é mais fácil pedir perdão do que permissão" merece cuidado em 2026. Hopper falava de burocracia interna, de pequenos experimentos que o chefe nunca aprovaria mas que davam certo. Aplicada a um agente de IA com acesso ao terminal, a mesma atitude pode apagar um banco de dados de produção. É possível imaginá-la ajustando o conselho: experimente sem pedir permissão no seu ambiente de testes, mas coloque o agente em uma *sandbox* (um ambiente isolado, como uma caixa de areia), com permissões mínimas e revisão humana antes de qualquer coisa irreversível. Ela, que era oficial da Marinha, entendia muito bem a diferença entre ousadia e falta de disciplina.

McCarthy e Minsky: o verão em que a IA ganhou nome

> **Para começar**
>
> Em 1955, quatro pesquisadores pediram verba para um verão estudando como máquinas poderiam aprender e usar linguagem. Não resolveram o problema, mas deram nome a uma área: inteligência artificial.

John McCarthy (1927–2011) cunhou o termo na proposta de Dartmouth, escrita com Minsky, Rochester e Shannon, e criou a linguagem Lisp e a coleta de lixo. Marvin Minsky (1927–2016) construiu uma das primeiras máquinas de rede neural e, com Papert, publicou *Perceptrons* (1969), que mostrou limites das redes de uma camada. A proposta afirmava que todo aspecto da inteligência poderia ser descrito com precisão suficiente para uma máquina simulá-lo; as previsões de prazo erraram muito.

Especulação. McCarthy, defensor da IA baseada em lógica, se interessaria por modelos combinados com verificação formal. Minsky, que ironicamente ajudou a esfriar o interesse por redes neurais, teria motivos para uma reação mista. As redes profundas de hoje superam exatamente as limitações que *Perceptrons* apontou, porque têm muitas camadas. Já a *Society of Mind* dele combina muito bem com a moda de 2026: sistemas multiagente (Livro 2, Capítulo 4 (Agentes e MCP)), em que vários agentes especializados (um planejador, um programador, um revisor) cooperam para resolver uma tarefa. Dá para imaginá-lo dizendo, com seu humor provocador, que finalmente alguém construiu uma sociedade da mente, mas que esqueceu de ler o resto do livro, que falava de emoções, de senso comum e de como os agentes discordam entre si.

Margaret Hamilton: quando o software não pode falhar

> **Para começar**
> Na descida da Apollo 11, o computador de bordo ficou sobrecarregado. Em vez de travar, largou as tarefas menos importantes e manteve as essenciais, e o pouso continuou.

Margaret Hamilton (nascida em 1936) dirigiu a equipe do MIT que escreveu o software de voo da Apollo e popularizou o termo "engenharia de software". Os alarmes 1202 e 1201 de 20 de julho de 1969 mostraram o valor do escalonamento por prioridade.

Especulação. Diante de um agente de programação, ela perguntaria o que acontece se o modelo gerar um comando destrutivo. Imagine-a liderando uma equipe em 2026. Ela usaria IA para gerar milhares de casos de teste e cenários de falha, deixaria agentes procurarem bugs no próprio código da equipe e exigiria que cada mudança gerada por IA passasse pelo mesmo processo de revisão de qualquer outra (Livro 3, Capítulo 4 (Desenvolvimento seguro e Pull Request)). E provavelmente teria uma regra simples: o código gerado por IA pode ir para a missão, mas o sistema precisa continuar seguro mesmo quando o modelo erra. Essa é a diferença entre um programa que funciona e um que é confiável.

Dennis Ritchie e Ken Thompson: a força das coisas pequenas

> **Para começar**
> O Unix desses dois programadores da Bell Labs é ancestral do Linux, do macOS e do Android, e a linguagem C sustenta sistemas operacionais e drivers de GPU. A filosofia era simples: programas pequenos, que fazem uma coisa bem e conversam por texto.

Thompson e Ritchie criaram o Unix a partir de 1969 e o reescreveram em C em 1973; receberam o Prêmio Turing em 1983. Thompson depois criou o UTF-8 com Rob Pike e a linguagem Go. No discurso "Reflections on Trusting Trust" (1984), ele mostrou que um compilador contaminado pode inserir uma porta dos fundos em programas e em novos compiladores, mesmo depois que o código malicioso é apagado: "*You can't trust code that you did not totally create yourself.*"

Especulação. A filosofia Unix também combina muito bem com os agentes atuais. Um agente que usa o terminal, encadeia `grep`, `sed` e `git` e lê a saída de texto de cada ferramenta está fazendo exatamente o que Thompson e Ritchie imaginaram: pequenas ferramentas compostas por um *pipe*. O protocolo MCP (Livro 2, Capítulo 4 (Agentes e MCP)) é, em certo sentido, uma versão moderna dessa ideia. Dá para imaginar Ritchie, que era discreto e preciso, olhando para um código em C gerado por IA e procurando primeiro os erros de manipulação de memória, a mesma classe de bugs que fez linguagens como Rust se popularizarem.

Especulação. O alerta de Thompson ficou ainda mais atual com modelos que podem esconder comportamentos ativados por gatilhos. A resposta provável deles não seria abandonar a IA, mas aplicar as defesas conhecidas: compilações reproduzíveis, revisão de dependências e cadeia de suprimentos assinada (Livro 4, Capítulo 4 (Engenharia Reversa e Segurança)).

Tim Berners-Lee e Geoffrey Hinton: os pioneiros que falam hoje

> **Para começar**
> Dois pioneiros desta lista estão vivos e dizem em voz alta o que pensam: o inventor da web quer os dados de volta com as pessoas, e um dos pais das redes neurais alerta para os riscos do que ajudou a criar.

Tim Berners-Lee (nascido em 1955) propôs a web no CERN em 1989, e a tecnologia foi liberada sem royalties em 1993. Hoje defende o **Solid**, em que cada pessoa guarda seus dados num "pod" e decide quais aplicativos podem acessá-los, e no livro *This Is for Everyone* (2025) argumenta que assistentes de IA deveriam trabalhar para o usuário, não para a plataforma.

Aqui a especulação é pequena, porque ele já descreveu o que quer. Dá para imaginar um assistente pessoal no estilo que ele defende: um agente de IA que roda no seu computador ou em um provedor que você escolhe, lê o seu pod Solid com seu histórico médico, suas compras e sua agenda, e negocia em seu nome com os serviços, sem que nenhuma empresa precise guardar uma cópia desses dados. Esse desenho junta o que este livro discutiu nos Livros 2 e 3 (RAG sobre os próprios documentos, IA local e segurança). E ele provavelmente apontaria a contradição: as redes sociais e os chatbots mais populares (Livro 5, Capítulo 5 (Redes sociais)) vivem exatamente do modelo de dados centralizado que ele quer desmontar.

Geoffrey Hinton (nascido em 1947) popularizou a retropropagação em 1986 e orientou os autores da AlexNet.

Em maio de 2023, Hinton deixou o Google, onde trabalhava desde 2013, dizendo que queria falar dos riscos da IA sem que isso fosse atribuído à empresa. Desde então, tem repetido alguns pontos: sistemas digitais podem aprender de um jeito mais eficiente que cérebros, porque milhares de cópias do mesmo modelo podem compartilhar instantaneamente o que aprenderam; não sabemos controlar sistemas mais inteligentes que nós; e existe um risco real, que ele estimou publicamente em cerca de 10% a 20%, de que a IA leve à extinção humana nas próximas décadas. Ele também se preocupa com riscos imediatos: desinformação, armas autônomas, ataques cibernéticos e a perda de empregos. No discurso do banquete do Nobel, em dezembro de 2024, pediu que governos obriguem as grandes empresas a dedicar uma parte muito maior de seus recursos à pesquisa de segurança. Yann LeCun, que dividiu com ele o Prêmio Turing, considera esses alertas exagerados (Livro 5, Capítulo 4 (Polêmicas)).

O que eles nos pediriam

> **Para começar**
> Reunidos numa sala, esses pioneiros não dariam uma resposta única. Mas alguns pedidos aparecem com insistência no que escreveram e viveram.

- **Entenda a máquina, não só o resultado.** Ada queria as relações por trás da música; Shannon mediu a informação em vez de admirá-la.
- **Projete para o erro.** Babbage caçava erros, Hamilton projetou para a sobrecarga, Thompson ensinou a desconfiar do compilador. Código gerado por IA é culpado até os testes provarem o contrário.
- **Deixe a máquina falar a nossa língua, sem abrir mão da precisão.** A responsabilidade pelo que a máquina faz continua com quem pediu.
- **Mantenha a tecnologia aberta e os dados com as pessoas.** Berners-Lee doou a web; o Unix se espalhou pelas universidades (Capítulo 1 (Comunidades e Linux)).
- **Leve os riscos a sério sem perder a coragem.** Ouse onde o erro é barato e seja rigoroso onde é caro.
- **Não deixe ninguém de fora.** Ada publicou com iniciais, Turing foi perseguido, Hopper e Hamilton trabalharam onde quase não havia mulheres.

Pioneiro	A pergunta que ele ou ela nos faria
Ada Lovelace	Quem é o autor do que a máquina cria, e quem recebe o crédito?
Charles Babbage	Qual é a taxa de erro, e como você mediu?
Alan Turing	O que exatamente você quer dizer com "pensar"?
John von Neumann	Para onde leva uma tecnologia que acelera a si mesma?
Claude Shannon	Quanta informação há, de fato, nessa resposta?
Grace Hopper	Por que vocês ainda fazem do jeito que sempre fizeram?
McCarthy e Minsky	Ela raciocina, ou só parece raciocinar?
Margaret Hamilton	O que acontece quando tudo dá errado ao mesmo tempo?
Ritchie e Thompson	Em que você está confiando sem ter verificado?
Tim Berners-Lee	De quem são os dados?
Geoffrey Hinton	Vocês sabem controlar algo mais inteligente que vocês?

Epílogo

Em 1843, uma jovem de 27 anos escreveu que uma máquina de engrenagens poderia um dia compor música. Em 1950, um matemático de 38 anos perguntou se máquinas poderiam pensar. Nenhum dos dois viu a resposta. Nós vivemos para vê-la, e temos nas mãos as máquinas com que eles sonharam e a responsabilidade que eles só puderam imaginar. Cada prompt, cada pull request revisado, cada escolha sobre quem pode usar, aprender e ser ouvido é uma nova linha nessa história. Ainda vemos só um pouco à frente. Mas há muito a ser feito, e agora sabemos fazer.

Fontes deste capítulo

Fatos atuais verificados em 31/10/2026.

Título	URL	Autor ou fonte	Ponto principal
Sketch of the Analytical Engine, with Notes by the Translator	https://www.fourmilab.ch/babbage/sketch.html	L. F. Menabrea e Ada Lovelace (1843)	Música por máquina e "não origina nada"
Passages from the Life of a Philosopher	https://www.gutenberg.org/ebooks/57532	Charles Babbage (1864)	A anedota dos "números errados"
Charles Babbage's Difference Engines and the Science Museum	https://www.sciencemuseum.org.uk/objects-and-stories/charles-babbages-difference-engines-and-science-museum	Science Museum	Construção da Máquina Diferencial nº 2 (1991)
Computing Machinery and Intelligence	https://academic.oup.com/mind/article/LIX/236/433/986238	A. M. Turing, Mind (1950)	Jogo da Imitação e máquinas que aprendem
The Chemical Basis of Morphogenesis	https://royalsocietypublishing.org/doi/10.1098/rstb.1952.0012	A. M. Turing (1952)	Padrões de reação-difusão na biologia
Large Language Models Pass the Turing Test	https://arxiv.org/abs/2503.23674	C. Jones e B. Bergen, UC San Diego (2025)	GPT-4.5 apontado como humano em 73%
Royal pardon for WW2 code-breaker Dr Alan Turing	https://www.gov.uk/government/news/royal-pardon-for-ww2-code-breaker-dr-alan-turing	Governo do Reino Unido (2013)	Perdão póstumo a Turing
First Draft of a Report on the EDVAC	https://web.mit.edu/STS.035/www/PDFs/edvac.pdf	John von Neumann (1945)	Arquitetura de programa armazenado
A Mathematical Theory of Communication	https://people.math.harvard.edu/~ctm/home/text/others/shannon/entropy/entropy.pdf	Claude E. Shannon (1948)	Entropia, bit e aproximações do inglês
Prediction and Entropy of Printed English	https://www.princeton.edu/~wbialek/rome/refs/shannon_51.pdf	Claude E. Shannon (1951)	0,6 a 1,3 bit por letra no inglês
A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence	http://jmc.stanford.edu/articles/dartmouth/dartmouth.pdf	McCarthy, Minsky, Rochester e Shannon (1955)	Origem do termo "inteligência artificial"
Margaret Hamilton	https://www.nasa.gov/people/margaret-hamilton-the-woman-who-saved-the-apollo-11-moon-landing/	NASA	Software de voo da Apollo e os alarmes 1202/1201
Reflections on Trusting Trust	https://www.cs.cmu.edu/~rdriley/487/papers/Thompson_1984_ReflectionsonTrustingTrust.pdf	Ken Thompson (1984)	O compilador contaminado e os limites da confiança
The Evolution of the Unix Time-sharing System	https://www.bell-labs.com/usr/dmr/www/hist.html	Dennis M. Ritchie	Origem do Unix
UTF-8 history	https://www.cl.cam.ac.uk/~mgk25/ucs/utf-8-history.txt	Rob Pike	Criação do UTF-8 em 1992
Information Management: A Proposal	https://www.w3.org/History/1989/proposal.html	Tim Berners-Lee (1989)	Proposta original da web
Tim Berners-Lee	https://www.w3.org/People/Berners-Lee/	W3C	Biografia e trabalho atual com o Solid
This Is for Everyone: The Unfinished Story of the World Wide Web	https://www.publishersweekly.com/9780374612467	Publishers Weekly	Livro de memórias de Berners-Lee (2025)
Evolving the web with Solid	https://theodi.org/news-and-events/blog/evolving-the-web-with-solid/	Open Data Institute	Pods de dados pessoais do Solid
The Nobel Prize in Physics 2024 (press release)	https://www.nobelprize.org/prizes/physics/2024/press-release/	Academia Real Sueca de Ciências	Nobel a Hopfield e Hinton
Learning representations by back-propagating errors	https://www.nature.com/articles/323533a0	Rumelhart, Hinton e Williams (1986)	Retropropagação em redes de várias camadas
A.M. Turing Award	https://amturing.acm.org/	ACM	Laureados citados no capítulo

Título	URL	Autor ou fonte	Ponto principal
Wikimedia Commons	https://commons.wikimedia.org/wiki/Main_Page	Wikimedia	Retratos em domínio público